

MISA: Memory-Efficient LLMs Optimization with Module-wise Importance Sampling

Yuxi Liu*
Peking University
yuxiliu666@stu.pku.edu.cn

Renjia Deng*
Peking University
2501210078@stu.pku.edu.cn

Yutong He
Peking University
yutonghe@pku.edu.cn

Xue Wang
Alibaba DAMO Academy
xue.wang@alibaba-inc.com

Tao Yao
Shanghai Jiao Tong University
taoyao@sjtu.edu.cn

Kun Yuan†
Peking University
kunyuan@pku.edu.cn

January 15, 2026

Abstract

The substantial memory demands of pre-training and fine-tuning large language models (LLMs) require memory-efficient optimization algorithms. One promising approach is layer-wise optimization, which treats each transformer block as a single layer and optimizes it sequentially, while freezing the other layers to save optimizer states and activations. Although effective, these methods ignore the varying importance of the modules within each layer, leading to suboptimal performance. Moreover, layer-wise sampling provides only limited memory savings, as at least one full layer must remain active during optimization. To overcome these limitations, we propose **Module-wise Importance Sampling (MISA)**, a novel method that divides each layer into smaller modules and assigns importance scores to each module. MISA uses a weighted random sampling mechanism to activate modules, provably reducing gradient variance compared to layer-wise sampling. Additionally, we establish an $\mathcal{O}(1/\sqrt{K})$ convergence rate under non-convex and stochastic conditions, where K is the total number of block updates, and provide a detailed memory analysis showcasing MISA’s superiority over existing baseline methods. Experiments on diverse learning tasks validate the effectiveness of MISA. Source code is available at: <https://github.com/pkumelon/MISA>.

1 Introduction

Large language models (LLMs) have emerged as a cornerstone of modern artificial intelligence, driving groundbreaking advancements in diverse domains such as machine translation, commonsense reasoning, and mathematical problem-solving. Despite their impressive capabilities, fine-tuning these models for downstream tasks introduces significant challenges [45, 46, 49], primarily due to the substantial memory overhead required to store optimizer states, gradients, and intermediate activations. These memory footprints are particularly

* Equal contribution.

† Corresponding author.

acute in resource-constrained environments, where the full-parameter fine-tuning of billion-scale models often exceeds the available hardware capacity, thereby impeding the practical deployment and scalability of LLMs [69, 60, 14].

To address these challenges, parameter-efficient fine-tuning (PEFT) methods, such as Low-Rank Adaptation (LoRA) [24] and its variants [35, 12], have gained considerable attention. These methods freeze the majority of pre-trained parameters and optimize only small, low-rank matrices integrated into transformer layers, leading to substantial reductions in memory usage. However, while these approaches are highly memory-efficient, they inherently constrain the model’s adaptability. By optimizing only a sparse subset of parameters, such methods often lead to suboptimal performance compared to full-parameter fine-tuning, as task-specific features crucial to downstream performance may reside within the frozen portions of the network [33, 66].

Recent research has explored block-coordinate descent (BCD) [55, 59, 62], a classical strategy for high-dimensional optimization, as a promising alternative for fine-tuning LLMs, resulting in the layer-wise optimization methods such as BAdam [37], HIFT [36], LIFT [75], LISA [41], and BlockLLM [47]. Unlike LoRA, these layer-wise methods optimize transformer blocks sequentially while keeping the remaining layers frozen. By iteratively updating all transformer blocks, layer-wise optimization effectively enables full-parameter updates, thereby preserving the expressive capacity of the original model. Empirical studies demonstrate that layer-wise optimization consistently outperforms LoRA in performance [37, 41]. Furthermore, by skipping gradient computations for frozen layers, layer-wise optimization eliminates the need to store their intermediate activations, offering greater memory efficiency than LoRA.

Motivating questions. All existing layer-wise LLM approaches [37, 36, 75, 41, 47] adopt the traditional BCD optimization paradigm, in which the model weights are divided into manageable blocks and updated iteratively. Although BCD provides a structured framework for handling the high dimensionality of LLM weights, fully realizing its potential in practical LLM fine-tuning requires addressing the following fundamental open questions.

Q1. How to effectively partition the LLM’s weights into blocks for optimization?

The choice of partitioning strategy can significantly impact optimization efficiency and convergence. Current methods [37, 36, 75, 41, 47] treat transformer layers as homogeneous units, overlooking the differing significance of internal modules—such as multi-head attention, feed-forward networks, and normalization layers—within each layer. By uniformly updating all parameters in a layer, these methods risk over-adapting less impactful modules while under-training critical ones, resulting in suboptimal optimization performance. Thus, more effective partitioning strategies must be explored.

Q2. How to effectively sample each block to achieve fast empirical convergence?

The performance of layer-wise methods is highly influenced by the block sampling strategies. However, current approaches primarily rely on cyclic [37] or uniform [36] sampling patterns, which overlook the varying importance across layers. LISA [41] keeps the Embedding and LLM head layers active but assigns equal sampling probabilities to all transformer layers, ignoring their varying significance. BlockLLM [47] prioritize more frequent updates to critical layers but ultimately focus on a small set of fixed blocks, failing to adequately explore the remaining layers. An effective sampling strategy should maintain a balance between thorough exploration of the parameter space and efficient exploitation of the most promising optimization directions to ensure rapid convergence.

Q3. How to establish convergence guarantees under practical LLM settings, incorporating Adam optimizer, stochastic gradients, and multiple updates per sampled block?

Theoretical convergence of layer-wise LLM optimization remains understudied. Traditional BCD optimization literature typically focuses on block-wise GD/SGD convergence [59, 61, 22]. Recent works [37, 74, 41] provide convergence guarantees under restrictive assumptions, such as noiseless gradients or single updates per sampled block. These analyses fail to reflect practical layer-wise LLM optimization, where Adam optimizer, stochastic gradients, and multiple updates per block are standard practice. A rigorous convergence analysis under these realistic conditions is essential for broader adoption of layer-wise optimization methods.

Main contributions. To address the aforementioned open questions, we propose a novel Module-wise Importance Sampling (**MISA**) method. MISA partitions the LLM’s weights into smaller modules, which serve as blocks for optimization. Additionally, we assign importance scores to each module and develop an effective strategy that dynamically samples modules based on real-time importance metrics. Our main contributions are summarized as follows:

- C1. Module-wise optimization.** We define a module as a matrix parameter within a transformer layer associated with weight gradients. Empirically, we observe that internal modules within transformer layers exhibit heterogeneous importance. Theoretically, we demonstrate that decomposing each layer into smaller modules preserves more information in gradient, thus motivating module-wise optimization. This addresses **Question 1**. Furthermore, we find this fine-grained update strategy eliminates the need to load a full layer into memory, making it more memory-efficient than layer-wise LLM optimization approaches [37, 36, 75, 41, 47].
- C2. Improved importance sampling.** Traditional sampling strategies often rely on heuristics, which can be suboptimal for LLM optimization. To improve this, we parameterize the gradient variance as a function of the sampling probability for each module and maximize it to optimize the sampling strategy. Additionally, we add a strategy to balance importance and uniform distributions, ensuring both comprehensive exploration of all modules and efficient exploitation of critical ones, addressing **Question 2**.
- C3. Convergence guarantees.** We demonstrate that MISA achieves a convergence rate of $\mathcal{O}(1/\sqrt{K})$, where K represents the total number of block updates. Our theoretical guarantees are derived under practical LLMs training scenarios, incorporating Adam optimization, stochastic gradients, and multiple updates per sampled block, thus addressing **Question 3**. Conventional BCD analysis relies on the assumption that block gradients are unbiased estimators of the full gradient—an assumption that fails when performing multiple updates within the same block. Our new analysis addresses this limitation by establishing fundamental connections between block-level gradients and the full gradient.

Table 1: Comparison of fine-tuning methods on LLaMA3-8B across eight commonsense reasoning tasks. The "ChatGPT" row presents results from ChatGPT obtained using Zero-shot CoT [58] with the GPT-3.5-turbo API. "Hella." refers to the HellaSwag dataset [65], and "Wino." refers to the Winogrande dataset [50]. Memory usage is reported without the application of additional memory-saving techniques such as gradient checkpointing or flash attention. The symbol δ denotes the proportion of parameters updated in each training iteration.

Model	Method	Mem.(GB)	BoolQ	PIQA	SIQA	Hella.	Wino.	ARC-e	ARC-c	OBQA	Avg.↑
ChatGPT	-	-	73.1	85.4	68.5	78.5	66.1	89.8	79.9	74.8	77.0
LLaMA3-8B	FT	150.5	75.1	89.2	80.4	96.2	88.3	92.4	82.5	89.8	86.7
	LoRA	35.7	70.8	85.2	79.7	92.5	84.9	88.9	78.7	84.4	82.5
	DoRA	54.1	74.6	89.3	79.9	95.5	85.6	90.5	80.4	85.8	85.2
	LISA	56.3	74.6	88.1	81.5	96	86.4	92.5	81.7	86.2	85.9
	BAdam	34.1	74.2	87.1	81.6	95	84.6	91.2	79.8	84.8	84.8
	MISA($\delta = 1\%$)	30.7	74.1	88.6	80.7	95	85.6	92	81	86.2	85.4
	MISA($\delta = 3\%$)	34.4	75.4	90.5	81.4	95.9	88.2	92.2	82.4	87	86.6

Experimental results. MISA demonstrates strong empirical performance.

E1. Fine-tuning. We evaluated MISA on different LLMs across three benchmarks: Commonsense Reasoning [25], Math Reasoning [25], and Instruction Following, encompassing a total of 16 datasets. We compared MISA against PEFT and layer-wise optimization methods, including LoRA [24], DoRA [35], BAdam [37], and LISA [41]. Under comparable memory constraints, MISA outperformed all baselines. An illustration of MISA’s superiority in fine-tuning tasks is provided in Table 1, with more detailed results presented in Section 5.

E2. Pre-training. We trained the LLaMA2 130M and 350M variant [32] on the C4 dataset [46]. In 350M model training, MISA achieved a perplexity of 22.11 after 2.7B training tokens, significantly outperforming GaLore’s 24.34 [71], and approaching Adam’s 21.3.

2 Related Work

Parameter-efficient fine-tuning. Adapter-based methods introduce new modules (e.g., fully connected layers) into the backbone model and update only these modules during training [43, 23, 19, 56, 20], incurring additional computational costs during inference. Prompt-based methods prepend random soft tokens to the input (typically as a prefix) and train their embeddings while keeping the backbone LLM frozen [31, 18, 73, 44], also introducing inference overhead.

LoRA and its variants, including QLoRA [13], DoRA [35], and AdaLoRA [68], have become state-of-the-art PEFT methods. LoRA injects low-rank matrices into linear layers to estimate weight updates while keeping the backbone LLM parameters frozen. Unlike adapter- and prompt-based methods, LoRA has no additional inference overhead, as the low-rank matrices can be merged into the model weights. To address the limitations of the low-rank structure, recent methods such as ReLoRA [32], MoRA [27], and HiRA [1] have explored high-rank updates based on LoRA.

Memory-efficient optimization. LOMO [39] eliminates optimizer and gradient memory costs during training with standard SGD. AdaLOMO [38] extends LOMO by integrating a fused backward operation with an adaptive learning rate per parameter. GaLore [71] enhances memory efficiency by projecting gradients into a low-rank subspace. Quantization techniques are widely applied to model parameters, gradients, optimizer states, and intermediate activations [11, 29, 10]. Adam-mini [70] reduces memory usage by optimizing learning rate allocation in Adam.

Layer-wise optimization. Inspired by classical Block Coordinate Descent (BCD), layer-wise optimization has emerged as a promising approach for fine-tuning large language models. Methods such as LIFT [75], HIFT [36], and BAdam [37] update LLM layers sequentially, achieving performance comparable to parameter-efficient fine-tuning (PEFT) methods while significantly reducing computational costs. LISA [41] identified a skewed weight-norm distribution across layers in LoRA and introduced a randomized layer update strategy, keeping the embedding and language modeling head layers activated. Similarly, BlockLLM [47] prioritizes frequent updates to layers with larger gradient norms. However, none of these approaches incorporate module-wise importance sampling.

Varying importance of modules in LLM fine-tuning. Previous studies have shown that different layers contribute unequally to LLM performance. LayerSkip [15], LayerDrop [16], and Layer-wise Model Pruning [17] demonstrated that the overparameterization of pre-trained models leads to layer redundancy, enabling high performance to be maintained even after removing certain layers. IST [64] and ILA [52] further highlighted that layer importance varies during LLM fine-tuning. Additionally, studies on PEFT have revealed that different modules within transformer blocks exhibit varying ranks [2, 68]. OwLore[30] identifies layer importance by detecting outliers in the model weights. However, these insights have not yet motivate module-wise importance sampling in LLM optimization.

3 Module-wise Importance Sampling

3.1 Problem Formulation

We consider the following problem:

$$\min_{\theta \in \mathbb{R}^d} \mathbb{E}_{\xi}[F(\theta; \xi)] \quad (1)$$

where $F(\theta; \xi)$ is a loss function that depends on a random variable ξ , and θ represents the set of d trainable parameters. Conventional algorithms update all elements of θ simultaneously, leading to high memory and computational costs that hinder the training of LLMs on low-end hardware. In this work, we propose an alternative formulation of the problem in (1). Specifically, we introduce a block-wise representation of θ , defined as $\theta = (\theta_1, \theta_2, \dots, \theta_B)$, where $\theta_b \in \mathbb{R}^{d_b}$ represents the b -th block of weights. Here, we have $d = \sum_{b=1}^B d_b$. The resulting problem formulation is then given by:

$$\min_{\theta_1, \dots, \theta_B} f(\theta_1, \theta_2, \dots, \theta_B) := \mathbb{E}_{\xi}[F(\theta_1, \theta_2, \dots, \theta_B; \xi)].$$

3.2 Block Sampling Strategy

At each time step, block-wise LLM training updates only a subset of parameters $\{\theta_i\}$. This approach inherently reduces the number of trainable parameters, resulting in lower memory and computational requirements. However, it also introduces greater variability in the gradient’s unbiased estimator, which can potentially hinder convergence performance. This section will derive an effective importance sampling strategy to enhance block-coordinate optimization.

Block coordinate descent. Let $g_b^n := [\nabla f(\theta^n)]_b \in \mathbb{R}^{d_b}$ denote the b -th block gradient of $\nabla f(\theta^n)$, where $b \in [B]$ is the block index and $n \geq 0$ is the iteration index. Additionally, we define the matrix $U_b := [0; \dots; I_b; \dots; 0] \in \mathbb{R}^{d \times d_b}$, where the b -th block of U_b is the identity matrix $I_b \in \mathbb{R}^{d_b \times d_b}$, and all other blocks are zero. Suppose each block gradient g_b^n is sampled with probability p_b , the BCD algorithm will iterate as follows

$$\theta^{n+1} = \theta^n - \alpha U_b g_b^n, \quad \text{where block } b \text{ is sampled with probability } p_b.$$

where α is the learning rate. Assume $f(\theta)$ is L -smooth, we have

$$\begin{aligned} f(\theta^{n+1}) &= f(\theta^n - \alpha U_b g_b^n) \leq f(\theta^n) - \alpha \langle \nabla f(\theta^n), U_b g_b^n \rangle + \frac{L\alpha^2}{2} \|U_b g_b^n\|^2 \\ &\stackrel{(a)}{=} f(\theta^n) - \alpha \left(1 - \frac{L\alpha}{2}\right) \|g_b^n\|^2 \stackrel{(b)}{\leq} f(\theta^n) - \alpha \|g_b^n\|^2 / 2 \end{aligned}$$

where (a) holds because $g_b^n = U_b^\top \nabla f(\theta^n)$ and (b) holds when $\alpha \leq 1/L$. Taking expectation on b ,

$$\mathbb{E}[f(\theta^n) - f(\theta^{n+1})] \geq \frac{\alpha}{2} \sum_{b=1}^B p_b \|g_b^n\|^2.$$

Importance sampling. An ideal update θ^{n+1} should maximize the expected decrease $\mathbb{E}[f(\theta^n) - f(\theta^{n+1})]$, ensuring the most significant descent in the objective function. This can be achieved by

$$\max_{\{p_b\}_{b=1}^B} \sum_{b=1}^B p_b \|g_b^n\|^2, \quad \text{s.t.} \quad \sum_{b=1}^B p_b = 1, \quad p_b \geq 0.$$

Intuitively, this strategy prioritizes blocks with larger expected gradient norms, as they contribute more significantly to optimization progress. However, excessively prioritizing important blocks can lead to a scenario where only a few blocks are updated frequently. This results in over-exploitation at the expense of exploration. To encourage broader exploration, we constrain the sampling probabilities to stay close to a uniform distribution by incorporating a Kullback–Leibler (KL) divergence penalty:

$$\max_{\{p_b\}_{b=1}^B} \sum_{b=1}^B p_b \|g_b^n\|^2 - (1/\eta) \text{KL}(p_b, q_B), \quad \text{s.t.} \quad \sum_{b=1}^B p_b = 1, \quad p_b \geq 0, \quad (2)$$

where $q_B = 1/B$ denotes the uniform distribution, and $\eta > 0$ is a coefficient controlling the trade-off between exploration and exploitation. As $\eta \rightarrow 0$, the KL divergence penalty dominates, and each p_b approaches uniform sampling; as $\eta \rightarrow +\infty$, the penalty vanishes, recovering standard importance sampling. The following proposition provides a closed-form solution to problem (2).

Proposition 1. *The optimal solution to problem (2) is given as follows*

$$p_b^n = \frac{\exp(\eta \|g_b^n\|^2)}{\sum_{b=1}^B \exp(\eta \|g_b^n\|^2)}, \quad (3)$$

which is the sampling probability of block b at iteration n .

Practical implementation. The sampling probability in (3) is impractical to implement, as $g_b^n = [\nabla f(\theta^n)]_b$ represents a full-batch block gradient, which is inaccessible during LLM optimization. In block-wise LLM training, each sampled block is typically updated for T steps using the Adam optimizer before switching to another block. Let $g_b^{n,t} := [\nabla F(\theta^{n,t}; \xi^{n,t})]_b$ denote the b -th block stochastic gradient at outer iteration n and inner update t . We approximate the full-batch gradient norm $\|g_b^n\|^2$ using the empirical average

$\frac{1}{T} \sum_{t=1}^T \|g_b^{n,t}\|^2$, resulting in a practical sampling strategy:

$$p_b^n = \frac{\exp(\eta G_b^n)}{\sum_{b=1}^B \exp(\eta G_b^n)} \quad \text{where} \quad G_b^n = \begin{cases} \beta G_b^{n-1} + (1-\beta) \frac{1}{T} \sum_{t=1}^T \|g_b^{n,t}\|^2 & \text{If } b \text{ is sampled;} \\ G_b^{n-1} & \text{otherwise.} \end{cases} \quad (4)$$

Instead of relying solely on the most recent T mini-batch block stochastic gradients, G_b^n aggregates all historical stochastic gradients to approximate the full-batch block gradient norm $\|g_b^n\|^2$ in (4). To eliminate the impact of differences in the number of parameters across blocks on gradient norm calculation, we actually use the **scaled gradient norm** in practice rather than the original gradient norm, and the specific definition is provided in Appendix A.2. We thus address **Question 2** by answering how to effectively sample blocks.

Remark 1 (MEMORY AND COMPUTATION OVERHEAD). *The computation and storage cost to maintain $\{G_b^n\}_{b=1}^B$ is negligible compared to that of the block stochastic gradients. See Sections F.3 and E.5 for details.*

Remark 2 (CLARIFICATION ON "LAYER", "MODULE", AND "BLOCK"). *A layer is a standard transformer component (e.g., with MHA and FFN), a module is a fine-grained subcomponent within a layer (e.g., W_q , W_k , W_v), and a block is a flexible optimization unit in block coordinate descent that can be a layer, a module, or a group of modules.*

3.3 Partition the Weights into Fine-Grained Modules Instead of Coarse Layers

Current approaches [37, 36, 75, 41, 47] often treat transformer layers as homogeneous structures, neglecting the distinct roles and contributions of their internal components—such as multi-head attention mechanisms, feed-forward networks, and normalization layers. As shown in Fig. 1, different modules within the same transformer layer exhibit significantly varying gradient norms, underscoring their differing levels of importance. Uniformly updating all modules within a layer risks over-adapting less critical components while under-training more essential ones, ultimately leading to suboptimal performance.

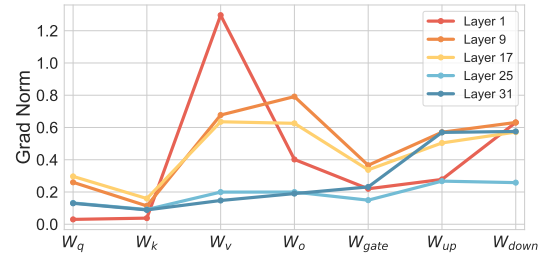


Figure 1: The gradient norm of different modules in different layers when fine-tuning LLaMA3-8B.

Motivated by this observation, we propose decomposing layers into distinct internal modules. In this work, a module is defined as a matrix parameter within a transformer layer that is associated with a weight gradient. For example, in the multi-head attention mechanism, we identify four modules— W_q , W_k , W_v , and W_o —while in the feed-forward network, we define two modules— W_{up} and W_{down} . In other words, we treat each module as a sampling block, addressing **Question 2**. Proposition 2 characterizes the advantages of module-wise over layer-wise block sampling.

Proposition 2. *Suppose θ_b in problem (2) contains K modules such that $\theta_b = (\theta_{b,1}, \theta_{b,2}, \dots, \theta_{b,K})$ which associates with block gradient $g_b = (g_{b,1}, g_{b,2}, \dots, g_{b,K})$. We further introduce p_{bj} as the sampling probability of the block gradient $g_{b,j}$. Under this setting, problem (2) transforms into*

$$\max_{\{p_{bj}\}} \sum_{b=1}^B \sum_{j=1}^K p_{bj} \|g_{b,j}^n\|^2 - (1/\eta) \text{KL}(p_{bj}, q_{BK}), \quad \text{s.t.} \quad \sum_{b=1}^B \sum_{j=1}^K p_{bj} = 1, \quad p_{bj} \geq 0, \quad (5)$$

Here, $q_{BK} = 1/(BK)$ is the uniform distribution. Any layer-wise importance sampling (i.e., a solution to problem (2)) is also a feasible solution to (5). Consequently, module-wise importance sampling (i.e., the optimal solution to problem (5)) will yield a larger objective function value than layer-wise importance sampling, making it a superior strategy to layer-wise sampling.

3.4 Module-wise Importance Sampling (MISA) Method

Based on the discussions in Sec. 3.2 and 3.3, we now introduce **Module-wise Importance Sampling (MISA)**. The pseudocode for MISA is provided in Algorithm 1. MISA is a double-loop algorithm: the outer loop,

indexed by n , selects a module, while the inner loop (Lines 7-12) updates the parameters of the selected module T times using the Adam optimizer, with each update indexed by t . As shown in Line 1, MISA partitions the model into modules rather than layers. Lines 14-15 illustrate that MISA follows the effective sampling strategy described in Proposition 1. By adjusting η , we can balance exploration across all modules with exploitation of the more important ones. In Line 16, the module weight $\theta_{\tau_n}^{n,T}$ undergoes an additional Adam step to obtain $\theta_{\tau_n}^{n+1,0}$, which facilitates the convergence analysis. In Line 17, we clear the optimizer states to ensure consistent memory efficiency throughout the training process.

Algorithm 1 Module-wise Importance Sampling (MISA)

Require: $\theta^0, N, T, B, \eta, \alpha, \delta > 0, \beta_1, \beta_2 \in (0, 1)$ and the sampling block τ_n at outer loop n

- 1: Partition the model into B modules (not layers); ▷ Partition weights into modules;
 - 2: Initialize probability weights $P^1 = (\frac{1}{B}, \dots, \frac{1}{B})$;
 - 3: Initialize the module gradient estimate $G_b^0 = 0$ for $b \in [B]$ and let $G^0 = (G_1^0, \dots, G_B^0)$;
 - 4: **for** $n = 1, \dots, N$ **do**
 - 5: Sample L_n modules (labeled with index τ_n) according to P^n such that the ratio of trainable parameter is less than δ . (See Algorithm 2 for more details); ▷ Importance sampling;
 - 6: Initialize $m_{\tau_n}^{n,0} = 0, v_{\tau_n}^{n,0} = 0$
 - 7: **for** $t = 1, \dots, T$ **do**
 - 8: Sample a batch of data and calculate block stochastic gradient $g_{\tau_n}^{n,t}$ for selected module τ_n ;
 - 9: Update the corresponding first-order and second-order momentum as follows:
 - 10: $m_{\tau_n}^{n,t} \leftarrow \beta_1 m_{\tau_n}^{n,t-1} + (1 - \beta_1) g_{\tau_n}^{n,t}, \quad v_{\tau_n}^{n,t} \leftarrow \beta_2 v_{\tau_n}^{n,t-1} + (1 - \beta_2) (g_{\tau_n}^{n,t})^2$
 - 11: Update the corresponding module as follows:
 - 12: $\theta_{\tau_n}^{n,t} \leftarrow \theta_{\tau_n}^{n,t-1} - \alpha m_{\tau_n}^{n,t} / (\sqrt{v_{\tau_n}^{n,t}} + \varepsilon)$
 - 13: **end for**
 - 14: Update G_b^n for each $b \in [B]$ according to (4); ▷ Track block gradient norm;
 - 15: Update $p_b^{n+1} \leftarrow \frac{\exp(\eta G_b^n)}{\sum_{j=1}^B \exp(\eta G_j^n)}$ for each $b \in [B]$; ▷ Update sampling probability;
 - 16: $\theta_{\tau_n}^{n+1,0} \leftarrow \theta_{\tau_n}^{n,T} - \alpha \frac{\beta_1}{1 - \beta_1} \frac{m_{\tau_n}^{n,T}}{\sqrt{v_{\tau_n}^{n,T}} + \varepsilon};$
 - 17: $g_{\tau_n}, m_{\tau_n}, v_{\tau_n} \leftarrow \text{None}$ ▷ Clear optimizer states;
 - 18: **end for**
 - 19: **Return** θ^N, P^N
-

Table 2: Comparison of existing memory-efficient optimization methods of LLMs.

	LoRA[24]	IST[64]	GaLore[71]	OwLore[30]	BAdam[37]	LISA[41]	MISA
Full-rank Update	✗	✗	✗	✗	✓	✓	✓
Importance-aware	✗	✓	✗	✓	✗	✓ ¹	✓
Fine-grained memory	✓	✓	✓	✓	✗	✗	✓
Gradient accumulation	✓	✓	✗	✗	✓	✓	✓
w/o SVD	✓	✓	✗	✗	✓	✓	✓
Fine-Tuning	✓	✓	✓	✓	✓	✓	✓
Pre-Training	✗	✗	✓	✓	✓	✓	✓
Convergence guarantee	✗	✗	☹	✗	☹	✗	☺

¹: LISA’s importance-aware strategy focuses only on the embedding layer and LM-head layer, while transformer layers which account for the majority of model parameters still use uniform sampling.

☹: GaLore’s convergence proof[71] holds only when the gradient exhibits a highly specialized structure, and BAdam’s analysis[37] applies exclusively to the full-gradient regime. Their settings in the theoretical guarantees rely on restrictive assumptions.

☺: MISA’s framework is based on the standard assumptions for analyzing non-convex optimization problems, making it more relevant to practical applications.

Table 2 provides a detailed comparison of existing optimization methods of LLMs. Notably, although LISA finds the embedding and LM-head layers to be very important, MISA does not train them in fine-tuning

tasks because their parameters are too large (proportional to vocabulary size), and training them would lead to a significant increase in memory consumption.

3.5 Memory Analysis

We perform detailed memory analysis of MISA and present a comparative overview of memory consumption between block-wise optimization and subspace optimization methods, such as LoRA and Galore. Our findings demonstrate MISA’s significant potential for long-sequence fine-tuning tasks. As illustrated in Fig. 2, for the LLaMA3-8B model, MISA significantly outperforms LoRA in terms of memory efficiency when the sequence length becomes sufficiently large. Notably, when $r = 16$, the trainable parameters in LoRA constitute only about 0.5% of the total parameters. In Appendix E, we demonstrate that (1) Layer-wise method achieves lower peak memory than subspace method such as LoRA and GaLore in the long-sequence setting. (2) Layer-wise method can update more weights than LoRA in memory-constrained scenarios. (3) MISA achieves greater memory efficiency compared to the layer-wise method when the sampling ratio threshold δ is small.

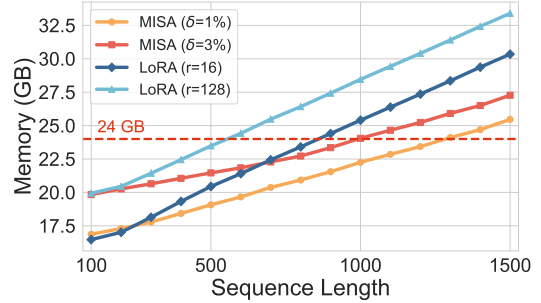


Figure 2: Comparison of peak memory consumption using MISA and LoRA fine-tuning on LLaMA3-8B across various sequence length.

4 Convergence Analysis

This section presents the convergence analysis for the proposed MISA method.

Limitations in existing analysis. Conventional BCD methods focus on the convergence of block-wise GD or SGD [59, 61, 22], but they do not generalize to the Adam optimizer. More recent studies [37, 74, 41] establish convergence results for layer-wise Adam, but only under restrictive conditions—such as assuming noiseless gradients or allowing just a single update per sampled block. These assumptions, however, diverge significantly from practical LLM training, where Adam is used with stochastic gradients and multiple updates are performed for each sampled block.

New challenges. Conventional BCD methods rely on the assumption that block gradients are unbiased estimators of the full gradient, an assumption that holds only when a single update is performed per sampled block—due to the preservation of unbiasedness through random block selection. However, our repeated block updating strategy introduces two key challenges: (1) Biased Gradient Estimates. Performing multiple updates on the same block breaks the randomness of block selection assumed in conventional BCD, resulting in biased estimates of the full gradient. (2) Bias-Noise Interplay. The introduced gradient bias interacts with inherent gradient noise, potentially amplifying estimation errors that would otherwise be mitigated by unbiased estimators.

Analysis highlights. Our work tackles these challenges via three innovations: (1) Bias Propagation Analysis. We derive novel recursive relations (Corollary 2) that characterize how gradient bias and stochastic noise accumulate over successive block updates. (2) Bridging Block and Full Gradients. We establish fundamental connections between block-level gradients and the full gradient (Corollary 3), despite the loss of unbiasedness. The additional Adam step in Line 15 of Algorithm 1 plays a critical role in ensuring a smooth transition from local block updates to the optimization of global variables. (3) Convergence Framework. We develop new analytical tools to jointly control gradient bias and amplified stochastic noise, resulting in rigorous convergence guarantees (Theorem 1).

Main results. We theoretically justify the convergence of MISA as follows.

Theorem 1 (Informal). Assume the loss function is L -smooth, the stochastic gradient is unbiased and bounded, the gradient variance is upper bounded (see Appendix D for detailed assumptions). With an appropriate learning rate α , MISA achieves the following convergence rate (proof is in Appendix D):

$$\frac{1}{N} \sum_{n=1}^N \mathbb{E}[\|\nabla f(\theta^{n,0})\|^2] = \mathcal{O}\left(\frac{1}{\sqrt{NT}} + \frac{1}{N}\right)$$

where N and T denote the numbers of outer and inner-loop iterations, respectively.

Remark 3. According to Theorem 1, MISA converges for any number T of inner local block updates, which constitutes a key advantage of our analysis. In contrast, the analyses in [59, 61, 74] are limited to the case $T = 1$, while the analysis in [48, 5] requires T to be sufficiently large to ensure each block is adequately updated.

5 Experiments

This section evaluates the performance of MISA across various fine-tuning and pre-training tasks. All our experiments are conducted on RTX 4090 24GB. We also conducted detailed ablation experiments, as presented in Appendix C.

5.1 Commonsense Reasoning

Settings. To validate the efficiency of MISA, we fine-tuned the LLaMA3-8B [14] and Qwen2.5-7B [63] models on the Commonsense Reasoning tasks. The dataset consists of eight subsets: BoolQ [6], PIQA [3], SIQA [51], HellaSwag [65], WinoGrande [50], ARC [7], and OBQA [40]. Following the settings in [25], we combined the training data from all eight tasks into a single training set for fine-tuning and then evaluated each model separately on the eight test sets. The baselines included LoRA [24], DoRA [35], and BCD-based methods, namely LISA [41] and BAdam [37]. We also reported GPU memory consumption for each method to ensure a thorough comparison. No additional memory-saving techniques, such as checkpointing [4] or flash attention [9], were employed.

Table 3: This table compares fine-tuning methods on Qwen2.5-7B models across eight commonsense reasoning tasks. "Hella." refers to HellaSwag [65], and "Wino." to Winogrande [50]. The first row shows ChatGPT results from Zero-shot CoT [58] using the GPT-3.5-turbo API. **Refer to Table 1 for results of LLaMA3-8B.**

Model	Method	Mem.(GB)	BoolQ	PIQA	SIQA	Hella.	Wino.	ARC-e	ARC-c	OBQA	Avg.↑
Qwen2.5-7B	FT	140.5	75.3	91	81.4	96.2	91.5	96.1	89.1	93.4	89.3
	LoRA	34.2	75.3	90.1	80.8	96.2	86.7	96.3	89	91.4	88.2
	DoRA	55.6	74.9	90.4	81.2	96.1	87.1	96.1	88.9	91.9	88.3
	LISA	57.9	75	90.3	80.6	95.2	86.2	95.8	89.1	89.8	87.8
	BAdam	33.7	74.5	90.3	80.8	95.5	86.1	96.1	89.6	91.8	88.1
	MISA($\delta = 1\%$)	30.3	74	90.1	82.4	95.5	87.1	96.1	89.3	90.2	88.1
	MISA($\delta = 3\%$)	34	75.4	90.5	81.6	95.7	87.2	96.3	89.5	92	88.5

Results. Table 3 presents the results for the commonsense tasks. Notably, DoRA’s additional memory consumption arises from activations, while LISA incurs extra memory usage from fine-tuning both the embedding layer and the language modeling head layer. MISA outperforms all baseline methods on both the LLaMA-3-8B and Qwen2.5-7B models. Furthermore, with the trainable parameter ratio set to 1%, MISA achieves performance comparable to LoRA while saving approximately 10% of memory. In particular, MISA demonstrates significant improvements over BAdam and LISA in the Qwen2.5-7B model.

5.2 Math Reasoning

Settings. To evaluate MISA on math reasoning tasks, we tested LLaMA3-8B and Qwen2.5-7B on four arithmetic benchmarks: GSM8K [8], SVAMP [42], AQUA [34], and MAWPS [28], following [25] for dataset settings. The models were fine-tuned on MATH10K [25], which combines training data from AQUA, GSM8K, and MAWPS, incorporating LM-generated chain-of-thought [58] steps.

Results. Table 4 presents the performance of various fine-tuning methods on LLaMA3-8B and Qwen2.5-7B. MISA outperforms all baselines on both models. Notably, MISA achieves a significant performance boost over LoRA and BAdam, which have similar GPU memory consumption. Compared to DoRA, MISA attains comparable performance with greater memory efficiency.

Table 4: Comparison of fine-tuning methods on LLaMA3-8B and Qwen2.5-7B across four arithmetic reasoning tasks. The left table presents results for LLaMA3-8B, while the right table displays results for Qwen2.5-7B. The unit of memory consumption is GB.

Method	Mem.	GSM8K	SVAMP	AQuA	MAWPS	Avg.↑	Mem.	GSM8K	SVAMP	AQuA	MAWPS	Avg.↑
LLaMA3-8B							Qwen2.5-7B					
LoRA	35.6	70.1	78.1	45.7	93.7	71.9	34.3	80.6	86.6	65.7	92.4	81.3
DoRA	54	70.9	78.4	50.3	93.4	73.3	54.8	80.1	86.7	68.1	92.5	81.9
LISA	54.2	68	76.7	49.7	88.3	70.7	55.6	77.6	86.8	70.1	89.8	81.1
BAdam	33.5	69.5	77.6	47.6	93.3	72	33.6	80.2	85.9	65.1	92.4	80.9
MISA($\delta = 1\%$)	30.4	68.5	77.3	49.3	90.1	71.3	29.8	77.5	85	65	89.5	79.3
MISA($\delta = 3\%$)	34.1	71.3	78.5	51.2	93.3	73.6	33.6	81	88.1	66.1	92.9	82

5.3 Instruction Tuning

To demonstrate MISA’s efficiency in instruction-following fine-tuning, we fine-tuned TinyLLaMA[67], LLaMA2-7B[54] and Mistral-7B[26] on the Alpaca GPT-4 dataset, which consists of 52K samples generated by GPT-4 based on inputs from the Alpaca dataset [53]. Model performance was evaluated on MMLU[21], MMLU-pro[57] and MT-Bench [72]. As shown in Table 5, MISA outperforms all baselines in most evaluation benchmarks, demonstrating its robustness and efficiency for instruction fine-tuning across different models. In addition, we compared the validation loss of MISA with other BCD methods, BAdam and LISA. As shown in Figure 3, the x-axis represents training time in minutes. MISA and LISA have similar total training time, while BAdam is faster than both. Despite this, MISA exhibits much better convergence compared to LISA and BAdam.

Table 5: Comparison of different methods on MMLU, MMLU-pro and MT-Bench. Results of LISA, GaLore, and LoRA in MMLU and MT-Bench are cited from LISA[41], while others are reproduced by us. MT-Bench evaluation was conducted using GPT-4. Memory was tested with batch size 2.

Model	Method	Mem.(GB)	MMLU (5-shot)	MMLU-pro (5-shot)	MT-Bench
TinyLLaMA	LoRA	5.00	25.81	11.32	1.90
	GaLore	5.00	25.21	10.96	2.61
	LISA	5.92	26.02	11.59	2.57
	BAdam	4.49	25.27	11.44	2.42
	MISA	4.36	25.40	11.65	2.73
LLaMA2-7B	LoRA	20.48	45.50	20.57	4.45
	GaLore	19.25	45.56	20.19	4.63
	LISA	22.79	46.21	20.85	4.94
	BAdam	19.83	45.61	20.68	4.81
	MISA	19.72	46.27	20.69	5.13
Mistral-7B	LoRA	21.96	61.78	32.96	4.41
	GaLore	20.87	57.87	31.87	4.36
	LISA	26.90	62.09	32.83	4.85
	BAdam	21.21	62.11	32.79	5.03
	MISA	21.18	62.90	33.44	5.19

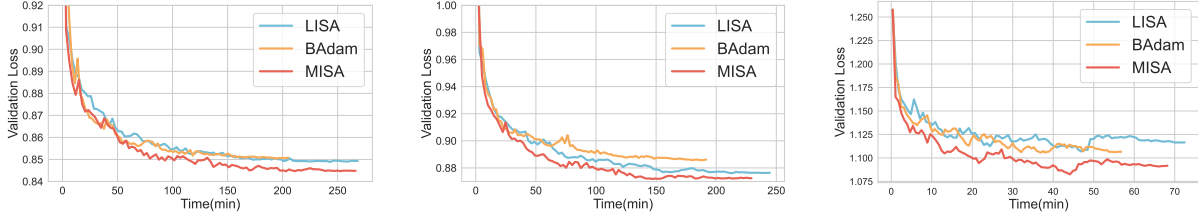


Figure 3: Validation loss of LISA, BAdam, and MISA across three epochs of fine-tuning Mistral-7B (left), LLaMA2-7B (middle) and TinyLLaMA (right) on the Alpaca-GPT4 dataset. The x-axis represents training time (minutes).

5.4 Pre-training

We evaluate MISA for pre-training LLaMA models [54] on the C4 dataset [46]. We pre-trained the 130M and 350M variant of LLaMA2 [33]. Unlike fine-tuning, where the embedding and output layers remain frozen, we trained them using the standard Adam optimizer. The number of training tokens is 2.75B. We report results for Adam, GaLore, and MISA. Table 6 reports the validation perplexity after 52K training steps. With a high-rank subspace (i.e., $r = 256$, $\delta = 25\%$), MISA outperforms GaLore and Adam on the 130M model. While on the 350M model, MISA outperforms GaLore and is slightly behind Adam. As shown in Fig. 4, MISA converges better than GaLore, and is similar with Adam’s performance in the later training stage. MISA’s strong performance indicates that it can be viewed as a regularization of Adam, and many gradients are redundant during full-parameter training.

Table 6: Validation perplexity of pre-training LLaMA 350M model on C4 dataset.

		Adam	GaLore		MISA	
			r=32	r=256	$\delta=3\%$	$\delta=25\%$
LLaMA 130M	Perplexity	24.63	44.88	28.27	36.07	23.81
	Memory(GB)	6.03	5.73	5.85	5.09	5.87
LLaMA 350M	Perplexity	21.30	39.43	24.34	30.62	22.11
	Memory(GB)	13.58	11.81	12.14	9.99	11.63

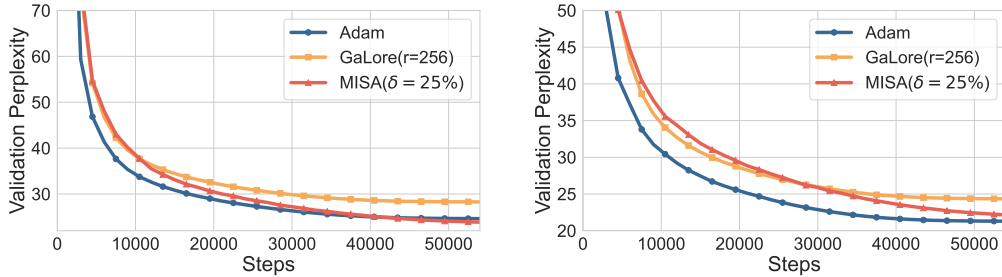


Figure 4: Pre-training dynamics for LLaMA 130M (left) and LLaMA 350M (right) on the C4 dataset.

6 Conclusion and Limitation

Conclusions. In this paper, we propose MISA, a novel memory-efficient training method. MISA decomposes each layer into smaller modules and employs an importance sampling mechanism for module training. We provide a detailed memory analysis demonstrating that MISA achieves high memory efficiency in long-sequence fine-tuning. Additionally, we show that MISA attains a convergence rate of $\mathcal{O}(1/\sqrt{K})$, where K is the total number of updates.

Limitations. Due to resource constraints, we pre-trained MISA at a small scale, and whether MISA can scale to large-scale LLMs pre-training (e.g., 7B, 70B, or larger models) remains to be validated. Additionally, MISA has only been verified on text-modal Transformer-based LLMs (e.g., LLaMA series, Qwen2.5 series, Mistral series) and has not been extended to multi-modal models or non-Transformer architectures, so its adaptability across different modalities and model structures needs further exploration.

7 Acknowledgements

Kun Yuan is supported by the National Natural Science Foundation of China (NSFC) under Grants W2441021, 92370121 and 12301392. Tao Yao is supported by the National Natural Science Foundation of China (NSFC) under Grants W2441021, 72371172, 72342023, and 71929101.

References

- [1] Anonymous. HiRA: Parameter-efficient hadamard high-rank adaptation for large language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [2] D. Biderman, J. Portes, J. J. G. Ortiz, M. Paul, P. Greengard, C. Jennings, D. King, S. Havens, V. Chiley, J. Frankle, et al. Lora learns less and forgets less. *arXiv preprint arXiv:2405.09673*, 2024.
- [3] Y. Bisk, R. Zellers, J. Gao, Y. Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.
- [4] T. Chen, B. Xu, C. Zhang, and C. Guestrin. Training deep nets with sublinear memory cost, 2016.
- [5] Y. Chen, Y. Zhang, Y. Liu, K. Yuan, and Z. Wen. A memory efficient randomized subspace optimization method for training large language models. *arXiv preprint arXiv:2502.07222*, 2025.
- [6] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*, 2019.
- [7] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafford. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [8] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [9] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [10] DeepSeek-AI. Deepseek-v3 technical report, 2024.
- [11] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer. 8-bit optimizers via block-wise quantization. *arXiv preprint arXiv:2110.02861*, 2021.
- [12] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023.
- [13] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 36, 2024.
- [14] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [15] M. Elhoushi, A. Shrivastava, D. Liskovich, B. Hosmer, B. Wasti, L. Lai, A. Mahmoud, B. Acun, S. Agarwal, A. Roman, A. Aly, B. Chen, and C.-J. Wu. Layerskip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, page 12622–12642. Association for Computational Linguistics, 2024.
- [16] A. Fan, E. Grave, and A. Joulin. Reducing transformer depth on demand with structured dropout, 2019.

- [17] C. Fan, J. Li, X. Ao, F. Wu, Y. Meng, and X. Sun. Layer-wise model pruning based on mutual information, 2021.
- [18] X. Han, W. Zhao, N. Ding, Z. Liu, and M. Sun. Ptr: Prompt tuning with rules for text classification. *AI Open*, 3:182–192, 2022.
- [19] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick, and G. Neubig. Towards a unified view of parameter-efficient transfer learning. *arXiv preprint arXiv:2110.04366*, 2021.
- [20] S. He, L. Ding, D. Dong, M. Zhang, and D. Tao. Sparseadapter: An easy approach for improving the parameter-efficiency of adapters. *arXiv preprint arXiv:2210.04284*, 2022.
- [21] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [22] M. Hong, X. Wang, M. Razaviyayn, and Z.-Q. Luo. Iteration complexity analysis of block coordinate descent methods. *Mathematical Programming*, 163:85–114, 2017.
- [23] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly. Parameter-efficient transfer learning for nlp. In *International conference on machine learning*, pages 2790–2799. PMLR, 2019.
- [24] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [25] Z. Hu, Y. Lan, L. Wang, W. Xu, E.-P. Lim, R. K.-W. Lee, L. Bing, and S. Poria. Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models. *arXiv preprint arXiv:2304.01933*, 2023.
- [26] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [27] T. Jiang, S. Huang, S. Luo, Z. Zhang, H. Huang, F. Wei, W. Deng, F. Sun, Q. Zhang, D. Wang, et al. Mora: High-rank updating for parameter-efficient fine-tuning. *arXiv preprint arXiv:2405.12130*, 2024.
- [28] R. Koncel-Kedziorski, S. Roy, A. Amini, N. Kushman, and H. Hajishirzi. Mawps: A math word problem repository. In *Proceedings of the 2016 conference of the north american chapter of the association for computational linguistics: human language technologies*, pages 1152–1157, 2016.
- [29] B. Li, J. Chen, and J. Zhu. Memory efficient optimizers with 4-bit states, 2023. URL <https://arxiv.org/abs/2309.01507>.
- [30] P. Li, L. Yin, X. Gao, and S. Liu. Owlcore: Outlier-weighted layerwise sampled low-rank projection for memory-efficient llm fine-tuning, 2024.
- [31] X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021.
- [32] V. Lialin, S. Muckatira, N. Shivagunde, and A. Rumshisky. Relora: High-rank training through low-rank updates. In *The Twelfth International Conference on Learning Representations*, 2023.
- [33] V. Lialin, N. Shivagunde, S. Muckatira, and A. Rumshisky. Relora: High-rank training through low-rank updates, 2023.
- [34] W. Ling, D. Yogatama, C. Dyer, and P. Blunsom. Program induction by rationale generation: Learning to solve and explain algebraic word problems. *arXiv preprint arXiv:1705.04146*, 2017.
- [35] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen. Dora: Weight-decomposed low-rank adaptation. In *Forty-first International Conference on Machine Learning*, 2024.
- [36] Y. Liu, Y. Zhang, Q. Li, T. Liu, S. Feng, D. Wang, Y. Zhang, and H. Schütze. Hift: A hierarchical full parameter fine-tuning strategy. *arXiv preprint arXiv:2401.15207*, 2024.
- [37] Q. Luo, H. Yu, and X. Li. Badam: A memory efficient full parameter optimization method for large language models. *Advances in Neural Information Processing Systems*, 37:24926–24958, 2024.

- [38] K. Lv, H. Yan, Q. Guo, H. Lv, and X. Qiu. Adalomo: Low-memory optimization with adaptive learning rate. *arXiv preprint arXiv:2310.10195*, 2023.
- [39] K. Lv, Y. Yang, T. Liu, Q. Gao, Q. Guo, and X. Qiu. Full parameter fine-tuning for large language models with limited resources. *arXiv preprint arXiv:2306.09782*, 2023.
- [40] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789*, 2018.
- [41] R. Pan, X. Liu, S. Diao, R. Pi, J. Zhang, C. Han, and T. Zhang. Lisa: layerwise importance sampling for memory-efficient large language model fine-tuning. *Advances in Neural Information Processing Systems*, 37:57018–57049, 2024.
- [42] A. Patel, S. Bhattamishra, and N. Goyal. Are nlp models really able to solve simple math word problems? *arXiv preprint arXiv:2103.07191*, 2021.
- [43] J. Pfeiffer, I. Vulić, I. Gurevych, and S. Ruder. Mad-x: An adapter-based framework for multi-task cross-lingual transfer. *arXiv preprint arXiv:2005.00052*, 2020.
- [44] G. Qin and J. Eisner. Learning how to ask: Querying lms with mixtures of soft prompts. *arXiv preprint arXiv:2104.06599*, 2021.
- [45] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang. Pre-trained models for natural language processing: A survey. *Science China technological sciences*, 63(10):1872–1897, 2020.
- [46] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.
- [47] A. V. Ramesh, V. Ganapathiraman, I. H. Laradji, and M. Schmidt. Blockllm: Memory-efficient adaptation of llms by selecting and optimizing the right coordinate blocks. *arXiv preprint arXiv:2406.17296*, 2024.
- [48] M. Razaviyayn, M. Hong, and Z.-Q. Luo. A unified convergence analysis of block successive minimization methods for nonsmooth optimization. *SIAM Journal on Optimization*, 23(2):1126–1153, 2013.
- [49] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve. Code llama: Open foundation models for code, 2024.
- [50] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- [51] M. Sap, H. Rashkin, D. Chen, R. LeBras, and Y. Choi. Socialliqa: Commonsense reasoning about social interactions. *arXiv preprint arXiv:1904.09728*, 2019.
- [52] G. Shi, Z. Lu, X. Dong, W. Zhang, X. Zhang, Y. Feng, and X.-M. Wu. Understanding layer significance in llm alignment, 2024.
- [53] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [54] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [55] P. Tseng. Convergence of a block coordinate descent method for nondifferentiable minimization. *Journal of optimization theory and applications*, 109:475–494, 2001.

- [56] Y. Wang, S. Agarwal, S. Mukherjee, X. Liu, J. Gao, A. H. Awadallah, and J. Gao. Adamix: Mixture-of-adaptations for parameter-efficient model tuning. *arXiv preprint arXiv:2205.12410*, 2022.
- [57] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- [58] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [59] S. J. Wright. Coordinate descent algorithms. *Mathematical programming*, 151(1):3–34, 2015.
- [60] L. Xu, H. Xie, S.-Z. J. Qin, X. Tao, and F. L. Wang. Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment. *arXiv preprint arXiv:2312.12148*, 2023.
- [61] Y. Xu and W. Yin. A block coordinate descent method for regularized multiconvex optimization with applications to nonnegative tensor factorization and completion. *SIAM Journal on imaging sciences*, 6(3):1758–1789, 2013.
- [62] Y. Xu and W. Yin. Block stochastic gradient iteration for convex and nonconvex optimization. *SIAM Journal on Optimization*, 25(3):1686–1716, 2015.
- [63] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang, G. Dong, H. Wei, H. Lin, J. Tang, J. Wang, J. Yang, J. Tu, J. Zhang, J. Ma, J. Xu, J. Zhou, J. Bai, J. He, J. Lin, K. Dang, K. Lu, K. Chen, K. Yang, M. Li, M. Xue, N. Ni, P. Zhang, P. Wang, R. Peng, R. Men, R. Gao, R. Lin, S. Wang, S. Bai, S. Tan, T. Zhu, T. Li, T. Liu, W. Ge, X. Deng, X. Zhou, X. Ren, X. Zhang, X. Wei, X. Ren, Y. Fan, Y. Yao, Y. Zhang, Y. Wan, Y. Chu, Y. Liu, Z. Cui, Z. Zhang, and Z. Fan. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- [64] K. Yao, P. Gao, L. Li, Y. Zhao, X. Wang, W. Wang, and J. Zhu. Layer-wise importance matters: Less memory for better performance in parameter-efficient fine-tuning of large language models, 2024.
- [65] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.
- [66] B. Zhang, Z. Liu, C. Cherry, and O. Firat. When scaling meets llm finetuning: The effect of data, model and finetuning method. *arXiv preprint arXiv:2402.17193*, 2024.
- [67] P. Zhang, G. Zeng, T. Wang, and W. Lu. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.
- [68] Q. Zhang, M. Chen, A. Bukharin, N. Karampatziakis, P. He, Y. Cheng, W. Chen, and T. Zhao. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning, 2023.
- [69] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.
- [70] Y. Zhang, C. Chen, Z. Li, T. Ding, C. Wu, D. P. Kingma, Y. Ye, Z.-Q. Luo, and R. Sun. Adam-mini: Use fewer learning rates to gain more, 2024.
- [71] J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar, and Y. Tian. Galore: Memory-efficient llm training by gradient low-rank projection. *arXiv preprint arXiv:2403.03507*, 2024.
- [72] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.
- [73] Z. Zhong, D. Friedman, and D. Chen. Factual probing is [mask]: Learning vs. learning to recall. *arXiv preprint arXiv:2104.05240*, 2021.
- [74] Y. Zhou, M. Zhang, J. Zhu, R. Zheng, and Q. Wu. A randomized block-coordinate adam online learning optimization algorithm. *Neural Computing and Applications*, 32(16):12671–12684, 2020.
- [75] L. Zhu, L. Hu, J. Lin, and S. Han. LIFT: Efficient layer-wise fine-tuning for large model models, 2024.

Appendix for “*MISA*: Memory-Efficient LLMs Optimization with Module-wise Importance Sampling”

Contents

1	Introduction	1
2	Related Work	4
3	Module-wise Importance Sampling	4
3.1	Problem Formulation	4
3.2	Block Sampling Strategy	5
3.3	Partition the Weights into Fine-Grained Modules Instead of Coarse Layers	6
3.4	Module-wise Importance Sampling (MISA) Method	6
3.5	Memory Analysis	8
4	Convergence Analysis	8
5	Experiments	9
5.1	Commonsense Reasoning	9
5.2	Math Reasoning	10
5.3	Instruction Tuning	10
5.4	Pre-training	11
6	Conclusion and Limitation	11
7	Acknowledgements	12
A	Algorithm Details	18
A.1	Module selection strategy	18
A.2	Definition of the scaled gradient norm	18
B	Additional Experiments	18
B.1	Memory Efficiency When Scaling To 70B Model	18
B.2	Enhancing LoRA with MISA	18
B.3	Computation Efficiency	19
C	Ablation Study	20
C.1	Ablation Study: Impact of Clearing Optimizer States	20
C.2	Ablation Study: Impact of Hyperparameters	20

C.3	Impact of Inner-Loop Iteration T	21
C.4	Ablation Study: Sampling Strategy	22
C.5	Ablation Study: Importance Scoring Methods	22
C.6	Ablation Study: Impact of Each Module	22
D	Theoretical Analysis.	24
D.1	Notation	24
D.2	Proof of theorem 1	34
E	Memory Analysis	35
E.1	Memory analysis of layer-wise optimization method	36
E.2	Memory analysis of MISA and LoRA	38
E.2.1	Memory analysis of Module-wise BCD	38
E.2.2	Memory analysis of LoRA	39
E.3	Memory analysis of GaLore	40
E.4	Memory analysis of MISA	40
E.5	Memory analysis of MISA’s indicators	41
E.6	Peak memory comparison between MISA and layer-wise method	41
E.7	Peak memory comparison between layer-wise method and LoRA	41
E.8	Paras/Peak-Memory comparison between layer-wise method and LoRA	42
F	Computation Analysis	42
F.1	Computation analysis of layer-wise optimization methods	44
F.2	Computation analysis of module-wise optimization methods	44
F.3	Computation analysis of MISA’s indicators	45
G	Broader Impacts	45
H	Experimental Hyperparameters	45
H.1	Hyperparameters for commonsense reasoning	45
H.2	Hyperparameters for math reasoning	46
H.3	Hyperparameters for instruction fine-tuning	47
H.4	Hyperparameters for pre-training	47

A Algorithm Details

A.1 Module selection strategy

Algorithm 2 Module selection

Require: Active set τ_n , Non-active set B , Sampling threshold δ , Probability distribution P^n , Total model parameters n_{model} , Counter S .

- 1: **Initialize:** $\tau_n = \emptyset$, $B = \{M_1, M_2, \dots, M_B\}$, $S = 0$
 - 2: **while** $B \neq \emptyset$ **do**
 - 3: Sample a module m from B based on the probability distribution P^n .
 - 4: $B = B - m$.
 - 5: **if** $S + |m| < \delta n_{model}$ **then**
 - 6: $\tau_n = \tau_n + t$
 - 7: **end if**
 - 8: **end while**
 - 9: **Return:** τ_n
-

A.2 Definition of the scaled gradient norm

For a given module m_i , we define its scaled gradient norm be the ratio of the Frobenius norm of the gradient to the square root of the number of parameters $\frac{\|g_i\|_F}{\sqrt{|m_i|}}$ in practice, which can be considered a definition of normalization.

B Additional Experiments

B.1 Memory Efficiency When Scaling To 70B Model

Fig. 5 demonstrates that when training a 70B model, MISA is still more memory-efficient than LoRA under long sequence length settings, and the use of flash-attention[9] has minimal impact on overall memory consumption.

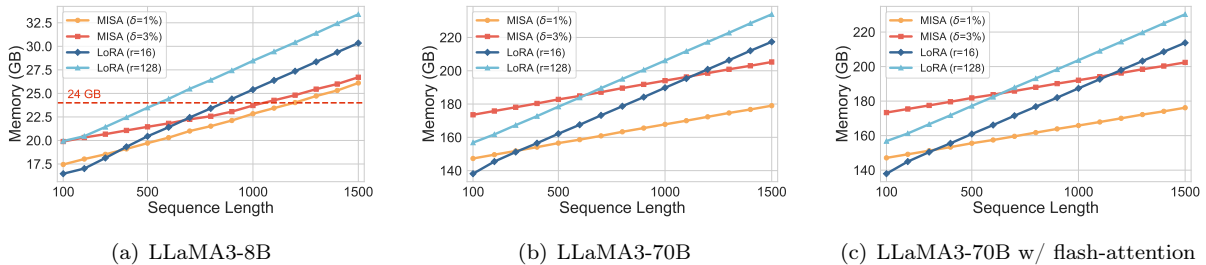


Figure 5: Comparison of peak memory consumption on LLaMA3-8B and LLaMA3-70B. (c) used flash-attention technique.

B.2 Enhancing LoRA with MISA

A key advantage of LoRA over MISA is its ability to train separate adapters for different downstream tasks. Instead of storing full model weights, LoRA only requires storage for task-specific adapters, significantly

reducing storage costs. Since MISA is orthogonal to LoRA and can be used alongside it, we explored a hybrid approach combining both methods. Given a model weight W with LoRA applied, it is represented as $W' = W + AB$, where A and B are treated as separate modules. We freeze the original model parameters and consider the LoRA adapters as the total set of trainable parameters n_{LoRA} . MISA dynamically activates LoRA adapters while ensuring that the number of active parameters at each step remains below $n_{\text{LoRA}} \times \delta$. Furthermore, we found that optimizer states are not the primary source of memory consumption for LoRA + MISA. Thus, we retain optimizer states for all LoRA adapters without clearing them. Our results show that MISA enhances full LoRA’s performance while further reducing memory usage. As shown in Fig. 6, activating no more than 30% of LoRA parameters achieves or even surpasses full LoRA’s performance while saving approximately 7.9% of memory. We also compared IST [64] with MISA (see Table 7) and MISA outperformed IST.

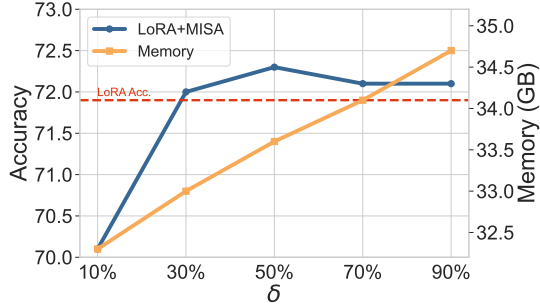


Figure 6: Accuracy and memory consumption of LoRA + MISA fine-tuning with varying δ on math reasoning tasks.

Table 7: Comparison of MISA and IST on LLaMA-7B and LLaMA3-8B across eight commonsense reasoning tasks. Results of LoRA+IST are cited from IST paper.

Model	Method	BoolQ	PIQA	SIQA	Hella.	Wino.	ARC-e	ARC-c	OBQA	Avg.↑
LLaMA-7B	LoRA	68.9	80.7	77.4	78.1	78.8	77.8	61.3	74.8	74.7
	LoRA+IST	68.7	81.7	77.3	82.7	78.7	80.6	62.4	80.0	76.5
	LoRA+MISA	70.4	81.5	78.9	83.4	78.9	81.2	63.1	79.7	77.1
LLaMA3-8B	LoRA	70.8	85.2	79.7	92.5	84.9	88.9	78.7	84.4	82.5
	LoRA+IST	72.7	88.3	80.5	94.7	84.4	89.8	79.9	86.6	84.6
	LoRA+MISA	74.4	88.9	81.8	95.3	86.1	91.3	81.4	86.5	85.7

B.3 Computation Efficiency

A training step includes forward propagation, backward propagation, and optimizer computations (including parameter updates and any algorithm-specific operations). Table 8 reports the average time per step for each method. GaLore incurs the highest optimizer computation overhead because it performs a full-model SVD (with very large time complexity) every 2000 steps. For MISA, we further conduct **detailed operation-level profiling** (under the same hardware and training configurations as Table 8) to quantify the overhead of its unique components (e.g., module sampling, gradient norm tracking). Results show that these components introduce negligible additional time (accounting for less than 0.05% of the total per-step time), as their key operations (e.g., gradient norm aggregation, sampling probability calculation) are either lightweight or amortized over multiple inner-loop steps. Thus, MISA’s computation efficiency is only behind BAdam’s, while it also achieves superior convergence with respect to training time (see Fig. 3).

Table 8: Comparison of training time for fine-tuning LLaMA2-7B on the Alpaca-GPT4 dataset. The Forward, Backward, and Optimizer time represent the average time per step. The last column shows the overall training time of 3 epochs.

Method	Forward(ms)	Backward(ms)	Optimizer(ms)	Averged cost per step(ms)
LoRA	70.3	104.5	4.5	179.3
GaLore	35.7	140.5	267.4	443.6
BAdam	34.3	42.9	1.5	78.7
LISA	36.4	84.1	2.9	123.4
MISA	35.2	74.6	1.8	111.6

C Ablation Study

C.1 Ablation Study: Impact of Clearing Optimizer States

In line 17 of algorithm1, we clear the optimizer state to save memory consumption. We explored a method to maintain the optimizer states of active layers during training. When a layer is frozen, its complete optimizer state is preserved and offloaded to CPU memory. These saved states are then precisely restored to GPU when the layer is reactivated in subsequent training cycles.

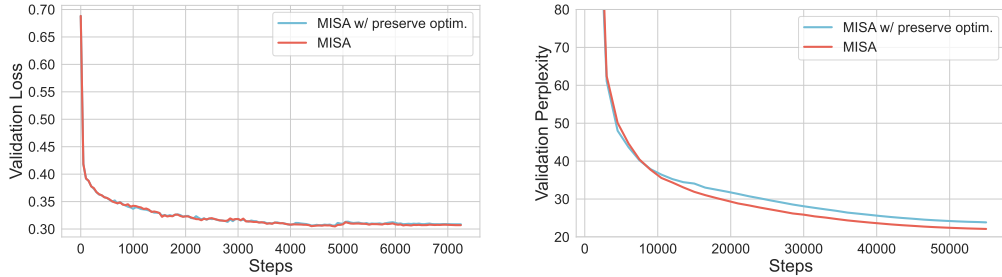


Figure 7: Fine-tuning loss (left) and pre-training perplexity (right) of LLaMA3-8B. Fine-tuning used MATH10K dataset, and pre-training used C4 dataset. MISA discards the optimizer states when switching activated layers, while "MISA w/ preserve optim." indicates retaining the optimizer states.

As shown in Fig. 7, preserving optimizer states has no significant impact on fine-tuning loss but brings significant extra pretraining perplexity. However, we don't have a particularly good explanation for this phenomenon. Our guess is that in the early stage of pre-training, when the model is still in the coarse-tuning stage, the dependence on the history of momentum is not that strong, so the clearing optimizer may be able to avoid local minima and thus improve optimization.

C.2 Ablation Study: Impact of Hyperparameters

We primarily investigated the effects of three hyperparameters: the learning rate, MISA's δ , and MISA's η . η controls the trade-off between importance sampling and uniform sampling. When $\eta = 0$, MISA reduces to uniform sampling. δ determines the number of parameters updated at each step and scales proportionally with memory consumption.

Fig. 8 presents the impact of η and δ . In practice, variations in η have a minor effect on model accuracy, whereas MISA is highly sensitive to the learning rate. An excessively large learning rate significantly degrades performance of the model.

Fig. 9 illustrates the validation loss over three epochs of fine-tuning LLaMA2-7B on the Alpaca-GPT4 dataset

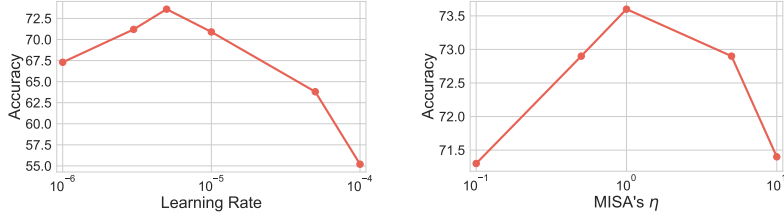


Figure 8: The accuracy of LLaMA3-8B on math reasoning tasks with different learning rate and η .

for different δ . We observe that larger δ lead to more severe overfitting in the third epoch, further supporting the interpretation of MISA as a regularization method of full parameter fine-tuning.

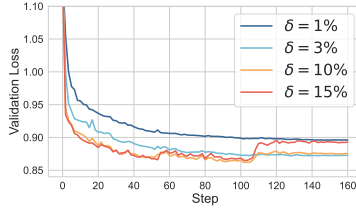


Figure 9: Validation loss of LLaMA2-7B on Alpaca-GPT4 dataset.

C.3 Impact of Inner-Loop Iteration T

We conduct an ablation study to evaluate the sensitivity of MISA to the inner-loop iteration count T (defined in Algorithm 1, representing the number of Adam updates for a sampled module before switching to another module). Experiments are conducted on the Mistral-7B model fine-tuned on the Alpaca-GPT4 dataset, with other hyperparameters fixed as: learning rate = $1e-5$, $\delta = 3\%$, $\eta = 0.5$, and batch size = 16. Table 9 reports the validation loss and MMLU score across different T values.

Table 9: Ablation Study of the Inner-Loop Iteration T for the Mistral-7B Model on Alpaca-GPT4

T	Validation Loss ($\downarrow$)	MMLU (5-shot, $\uparrow$)
5	0.877	46.22
15	0.874	46.23
30	0.871	46.17
50	0.873	46.27
100	0.877	46.19
200	0.881	46.01
500	0.879	45.89

As shown in Table 9, varying T from 5 to 500 does not significantly affect the convergence of validation loss or the MMLU performance. This observation aligns with the findings in BAdam [37] and LISA [41], confirming that MISA’s convergence is robust to the choice of T —a result supported by our theoretical analysis in Theorem 1, where MISA is proven to converge for any number of inner-loop updates T . Notably, when $T \geq 200$, we observe a slight degradation in MMLU score (e.g., 46.01 at $T = 200$ and 46.27 at $T = 50$), which may stem from over-fitting to the sampled module during extended inner-loop updates. Thus, we recommend setting $T = 30$ – 50 in practice to balance computational efficiency and performance.

C.4 Ablation Study: Sampling Strategy

To validate the necessity of MISA’s sampling strategy, we explored alternative approaches. The Uniform strategy samples modules randomly without computing importance scores, effectively replacing BAdam’s layer-wise optimization with module-wise optimization. The Top-K strategy selects the most important modules while ensuring that the total sampled parameters remain below δ of all parameters. In contrast, the Bottom-K strategy selects the least important modules under the same δ constraint. As shown in Table 10, the Uniform and Top-K strategies achieve similar performance but remain inferior to MISA, while the Bottom-K strategy significantly reduces accuracy. We observed that with the Top-K strategy, many modules are never sampled, which may explain its suboptimal performance. The effectiveness of MISA can be attributed to its balanced approach, combining elements of both the Uniform and Top-K strategies. It prioritizes modules with higher importance scores while ensuring that all modules have a chance to be sampled.

Table 10: Results of different sampling strategies for fine-tuning LLaMA3-8B on math and commonsense reasoning tasks. The reported scores indicate the average accuracy across all tasks.

Strategy	Math	Commonsense
MISA	73.6	86.6
Uniform	71.1(-2.5)	85.9(-0.7)
Top-K	71.2(-2.4)	86(-0.6)
Bottom-K	69.6(-4.0)	82.1(-4.5)

C.5 Ablation Study: Importance Scoring Methods

In (4), we identify each module’s importance score via its gradient norm. To validate its effectiveness, we also evaluated two alternative scoring functions: the module’s weight norm and its total parameter count. Table 11 compares those importance scoring methods and demonstrates that MISA’s sampling strategy significantly outperforms the alternatives, thereby validating the efficacy of MISA.

Table 11: Comparison of different scoring function in MISA. The accuracy of commonsense reasoning and math reasoning benchmarks are averaged across all tasks.

Model	Method	MMLU	MMLU-pro	Commonsense	Math
LLaMA3-8B	Weight Norm	64.5	35.9	85.7	71.9
	Number of Parameters	63.7	36.2	85.9	72.7
	MISA(Gradient Norm)	65	36.5	86.6	73.6

C.6 Ablation Study: Impact of Each Module

To assess the impact of each module, we applied MISA to individual modules while keeping all others frozen. Fig. 10 shows that the effectiveness of fine-tuning follows the order: $W_q, W_k < W_v < W_o, W_{gate}, W_{up}, W_{down}$. Interestingly, when applying MISA to all modules, the sampling frequency does not strictly follow this order, as shown in Fig. 11. We think there may be three reasons: (1) Sampling frequency does not directly reflect importance—more critical layers may require less fine-tuning for optimal performance. (2) When training multiple modules simultaneously, the combined importance of those modules may not be simply the sum of each module’s individual importance. (3) Module sizes vary significantly; for instance, $W_{gate}, W_{up}, W_{down}$ contain over ten times as many parameters as W_k and W_v . Small-size modules are less likely to exceed the sampling threshold δ , leading to more frequent sampling.

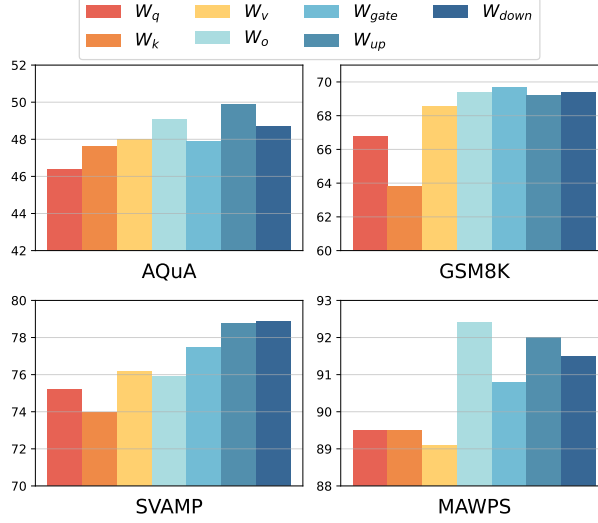


Figure 10: Accuracy comparison on math reasoning tasks when sample each individual module separately.

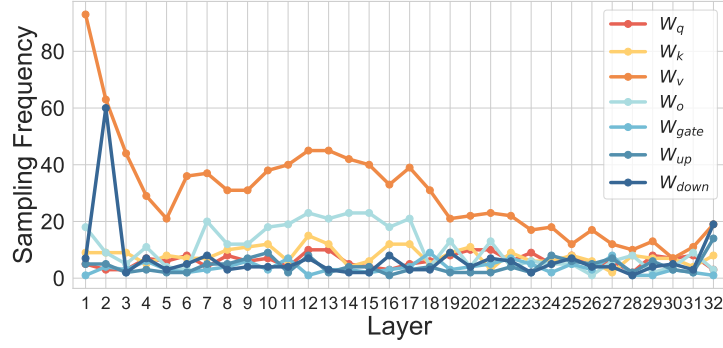


Figure 11: The frequency of module sampling across all layers when applying MISA to LLaMA3-8B.

Therefore, we further applied both uniform and MISA sampling strategies to individual modules in isolation, in order to assess which modules benefit most from MISA. As shown in Table 12, when fine-tuning W_q , W_{gate} , W_{down} and W_{down} , MISA achieves substantially larger accuracy gains than uniform sampling.

Table 12: Average accuracy of LLaMA3-8B on math reasoning benchmark when fine-tuning only 1 module type with uniform and MISA sampling strategies. Red indicates an improvement of more than 1%.

Model	Method	W_q	W_k	W_v	W_o	W_{gate}	W_{up}	W_{down}
LLaMA3-8B	Uniform	70	67.1	70.5	71.2	69.2	71.4	70.5
	MISA	69.5	68.8	70.5	71.7	71.5	72.5	72.1

D Theoretical Analysis.

D.1 Notation

- Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be the objective function to minimize.
- Let $f^* := \inf_{\theta \in \mathbb{R}^d} f(\theta)$ be the optimal value of f .
- Let ∇f denote the estimator for the gradient of f at the queried points.
- Let α denote the learning rate.
- Let n denote the block epoch, $n = 1, 2, \dots, N$.
- Let t denote the inner iterations for a certain block, $t = 0, 1, \dots, T$.
- Let $\xi^{n,t}$ denote the sampled data at n -th block epoch and t -th inner iteration.
- Let τ_n denote the sampled block at n -th block epoch.
- Let b denote the block's ID, $b = 1, 2, \dots, B$.
- Let $\theta_{\tau_n}^{n,t}$ denote the parameter when selected module τ_n at n -th block epoch and t -th inner adam step.
- Let $g_{\tau_n}^{n,t}$ denote the partial derivative conditioned on data $\xi^{n,t}$ at n -th block epoch and t -th inner adam step.
- Let $\tilde{G}_{\tau_n}^{n,T} = \frac{1}{T} \sum_{t=1}^T \|g_{\tau_n}^{n,t}\|^2$ be the cumulative gains during a sampling period.
- Let $\Gamma_{\tau_n}^{n,t} = \text{Diag}^{-1/2}(\tilde{v}_{\tau_n}^{n,t} + \varepsilon, \|v_{\tau_n}^{n,t-1}\|_{\min} + \varepsilon)$, $\tilde{v}_{\tau_n}^{n,t} = \max(v_{\tau_n}^{n,t}, \|\tilde{v}_{\tau_n}^{n,t-1}\|_{\max})$.
- Let $\theta^{n,t}$ denote the full parameter at n -th block epoch and t -th inner adam step.

$$\theta^{n,t} := \left(\theta_1^{n-1,T}, \theta_2^{n-1,T}, \dots, \theta_{\tau_n-1}^{n-1,T}, \theta_{\tau_n}^{n,t}, \theta_{\tau_n+1}^{n-1,T}, \dots, \theta_B^{n-1,T} \right).$$
- Let $\mathcal{F}_t$ denote the filtration containing all historical variables at and before computing $\theta_{\tau_n}^{n,t}$.

$$\mathcal{F}_t := \left\{ \theta_{\tau_n}^{n,t}, \xi^{n,t}, g_{\tau_n}^{n,t}, m_{\tau_n}^{n,t-1}, v_{\tau_n}^{n,t-1}, \theta_{\tau_n}^{n,t-2}, \dots, \xi^{n,1}, g_{\tau_n}^{n,1}, m_{\tau_n}^{n,0}, v_{\tau_n}^{n,0}, \theta_{\tau_n}^{n,0} \right\}$$
- Let $\mathbb{E}_t$ denote the expectation conditioned on $\mathcal{F}_t$.

To enhance the smoothness of the proof procedure, we utilized three techniques.

1. Second-order Momentum Inheritance

To establish Lemma 1, we utilize a second-order momentum inheritance technique. Upon completing the $(n-1)$ -th block epoch, we initialize the second-order momentum for the n -th block epoch as:

$$v_{\tau_n}^{n,0} = \|\tilde{v}_{\tau_n-1}^{n-1,T}\|_{\max} I \quad \text{for } n \geq 2$$

This initialization strategy maintains memory efficiency, with its additional overhead being negligible compared to gradient storage requirements.

2. Additional Momentum Step

The additional momentum step arises naturally from the variable transformation in (11). This carefully designed operation prevents momentum wastage in τ_n during the n -th block epoch, significantly simplifying the analysis at block transition points. The step is defined as:

$$\theta_{\tau_n}^{n+1,0} \leftarrow \theta_{\tau_n}^{n,T} - \alpha \frac{\beta_1}{1 - \beta_1} \frac{m_{\tau_n}^{n,T}}{\sqrt{v_{\tau_n}^{n,T} + \varepsilon}}$$

3. Single-module Gradient Propagation

While MISA typically activates multiple modules simultaneously, our theoretical analysis focuses on the single-module case for clarity. The multi-module scenario can be represented by introducing a diagonal projection matrix Q_n , transforming the stochastic gradient in Algorithm 3 to:

$$g_{L_n}^{n,t} = Q_n \left. \frac{\partial}{\partial \theta^{n,t-1}} f(\theta^{n,t-1}) \right|_{\xi^{n,t}}$$

where L_n denotes the set of activated modules at the n -th step. Our analysis naturally extends to this generalized case.

Algorithm 3 MISA:Analytical View

Require: $\theta^0, N, T, B, \eta, \alpha, \delta > 0, \beta_1, \beta_2 \in (0, 1)$ and the sampling block τ_n at outer loop n

- 1: Partition the model into B modules (not layers); ▷ Partition weights into modules;
- 2: Initialize probability weights $P^1 = (\frac{1}{B}, \dots, \frac{1}{B})$;
- 3: Initialize the module gradient estimate $G_b^0 = 0$ for $b \in [B]$ and let $G^0 = (G_1^0, \dots, G_B^0)$;
- 4: **for** $n = 1, \dots, N$ **do**
- 5: Sample a module τ_n according to P^n ; ▷ Importance sampling;
- 6: Initialize $m_{\tau_n}^{n,0} = 0, v_{\tau_n}^{1,0} = 0, v_{\tau_n}^{n,0} = \|\tilde{v}_{\tau_n-1}^{n-1,T}\|_{\max} I$ if $n \geq 2$;
- 7: **for** $t = 1, \dots, T$ **do**
- 8: Sample a batch of data $\xi^{n,t} \sim D$
- 9: Calculate block stochastic gradient $g_{\tau_n}^{n,t} = \left. \frac{\partial}{\partial \theta_{\tau_n}^{n,t-1}} f(\theta_{\tau_n}^{n,t-1}) \right|_{\xi^{n,t}}$ for selected module τ_n ;
- 10: Update the corresponding first-order and second-order momentum as follows:
- 11: $m_{\tau_n}^{n,t} \leftarrow \beta_1 m_{\tau_n}^{n,t-1} + (1 - \beta_1) g_{\tau_n}^{n,t}, \quad v_{\tau_n}^{n,t} \leftarrow \beta_2 v_{\tau_n}^{n,t-1} + (1 - \beta_2) (g_{\tau_n}^{n,t})^2$
- 12: Update $\tilde{v}_{\tau_n}^{n,t}$ as follows:
- 13: $\tilde{v}_{\tau_n}^{n,t} = \max(v_{\tau_n}^{n,t}, \|\tilde{v}_{\tau_n}^{n,t-1}\|_{\max})$ ▷ AMSGrad-type normalization;
- 14: Update the corresponding module as follows:
- 15: $\theta_{\tau_n}^{n,t} \leftarrow \theta_{\tau_n}^{n,t-1} - \alpha m_{\tau_n}^{n,t} / (\sqrt{\tilde{v}_{\tau_n}^{n,t}} + \varepsilon)$
- 16: **end for**
- 17: Update G_b^n for each $b \in [B]$ according to (4); ▷ Track block gradient norm;
- 18: Update $p_b^{n+1} \leftarrow \frac{\exp(\eta G_b^n)}{\sum_{j=1}^B \exp(\eta G_j^n)}$ for each $b \in [B]$; ▷ Update sampling probability;
- 19: $\theta_{\tau_n}^{n+1,0} \leftarrow \theta_{\tau_n}^{n,T} - \alpha \frac{\beta_1}{1 - \beta_1} \frac{m_{\tau_n}^{n,T}}{\sqrt{v_{\tau_n}^{n,T} + \varepsilon}}$; ▷ Additional momentum step;
- 20: $g_{\tau_n}, m_{\tau_n}, v_{\tau_n} \leftarrow \text{None}$ ▷ Clear optimizer states;
- 21: **end for**
- 22: **Return** θ^N, P^N

Assumption 1. The loss function is $\bar{L}$ -smooth. And when restricted on i -th block, it is L_i -smooth.

$$\|\nabla L(\theta^1) - \nabla L(\theta^2)\| \leq \bar{L} \|\theta^1 - \theta^2\|, \quad \left\| \frac{\partial L}{\partial \theta_i} \Big|_{\theta_i^1} - \frac{\partial L}{\partial \theta_i} \Big|_{\theta_i^2} \right\| \leq L_i \|\theta_i^1 - \theta_i^2\|, \quad i = 1, \dots, B.$$

Let $L = \max\{\bar{L}, L_i\}_{i=1, \dots, B}$.

Assumption 2. (Unbiased and bounded stochastic gradient). For all inner iterates $t \geq 1$ at n -th block epoch, the stochastic gradient $g_{\tau_n}^{n,t}$ is unbiased and uniformly bounded by some constant $G \geq 0$.

$$\mathbb{E}_t [g_{\tau_n}^{n,t}] = \nabla f(\theta_{\tau_n}^{n,t-1}), \quad \|g_{\tau_n}^{n,t}\| \leq G.$$

Assumption 3. (Bounded variance). For all inner iterates t at n -th block epoch, the variance of the stochastic gradient $g_{\tau_n}^{n,t}$ is uniformly bounded by some constant $\sigma^2 \geq 0$.

$$\mathbb{E}_t [\|g_{\tau_n}^{n,t} - \nabla f(\theta_{\tau_n}^{n,t-1})\|^2] \leq \sigma^2$$

Proposition 3. (Proof of proposition 1) Consider the optimization problem

$$\begin{aligned} \max_p \quad & \sum_{i=1}^B G_i^n p_i - \eta \text{KL}(p_i, q_B) \\ \text{s.t.} \quad & p_i \geq 0, \quad \sum_{i=1}^B p_i = 1, \end{aligned}$$

the solution is

$$p_i^{n+1} = \frac{\exp(\eta_0 G_i^n)}{\sum_{b=1}^B \exp(\eta_0 G_b^n)}$$

where $\eta_0 = \frac{1}{\eta}$.

Proof. We consider the equivalence problem

$$\max_{p^n} \eta_0 \sum_{i=1}^B G_i^n p_i - \text{KL}(p_i, q_B)$$

Where $\eta_0 = \frac{1}{\eta}$.

We construct the Lagrangian function $\mathcal{L}(P, \lambda)$ of the optimization problem

$$\mathcal{L}(P, \lambda) = \eta_0 \sum_{i=1}^B p_i G_i^n - \sum_{i=1}^B p_i \log \frac{p_i}{q_B} + \lambda \left(\sum_{i=1}^B p_i - 1 \right)$$

We take the partial derivative of p_i and obtain

$$\frac{\partial \mathcal{L}}{\partial p_i} = \log q_B - \log p_i - 1 + \eta_0 G_i^n + \lambda = 0, \quad i = 1, 2, \dots, B$$

Which is equivalent to

$$p_i = q_B \exp(\eta_0 G_i^n + \lambda - 1), \quad i = 1, 2, \dots, B.$$

According to the normalization condition, we have

$$1 = \sum_{i=1}^B p_i = \sum_{i=1}^B q_B \exp(\eta_0 G_i^n + \lambda - 1)$$

Let $Z = \exp(1 - \lambda)$, we have

$$Z = \sum_{i=1}^B q_B \exp(\eta_0 G_i^n)$$

Which implies

$$p_i^{n+1} = \frac{q_B \exp(\eta_0 G_i^n)}{\sum_{i=1}^B q_B \exp(\eta_0 G_i^n)} = \frac{\exp(\eta_0 G_i^n)}{\sum_{i=1}^B \exp(\eta_0 G_i^n)}$$

□

Proposition 4. (Proof of proposition 2) Suppose that in each layer we have K modules such that $\theta_i = (\theta_{i_1}, \theta_{i_2}, \dots, \theta_{i_K})$, the sampling probability of each layer be p_i , each module be $p_{i,j}$, the gain of each module be $G_{i,j}$ at n -th step and certain iteration, we have

$$\frac{1}{K} \sum_{i=1}^B G_i^n p_i \leq \sum_{i=1}^B \sum_{j=1}^K G_{i,j}^n p_{i,j}$$

Proof. We consider $p_i = \sum_{j=1}^K p_{i,j}$, which means the sum of each module's sampling probability is equal to the sampling probability of the corresponding layer.

Noting that in layer-wise setting, the sampling probability of each module is $\frac{p_i}{K}$.

Next, we will prove

$$G_i^n \frac{p_i}{K} \leq \sum_{j=1}^K G_{i,j}^n p_{i,j}$$

Noting that gain G_i^n is equal to the sum of each individual module gains $G_{i,j}^n$, we have

$$\frac{\sum_{j=1}^K G_{i,j}^n}{K} \leq \sum_{j=1}^K G_{i,j}^n \hat{p}_{i,j} \quad (6)$$

Where $\hat{p}_{i,j} = \frac{p_{i,j}}{p_i}$, $\sum_{j=1}^K \hat{p}_{i,j} = 1$.

By symmetry, we can assume without loss of generality that $G_{i,j}^n$ is ordered as follows.

$$G_{i,1}^n \leq G_{i,2}^n \leq \dots \leq G_{i,K}^n$$

From Proposition 3, it's obvious that

$$\hat{p}_{i,1} \leq \hat{p}_{i,2} \leq \dots \leq \hat{p}_{i,K}$$

The left side of inequality (6) is arithmetic average, the right side is weighted average of $G_{i,j}^n$.

From Chebyshev's inequality, inequality (6) holds. Summing over i , we obtain

$$\frac{1}{K} \sum_{i=1}^B G_i^n p_i \leq \sum_{i=1}^B \sum_{j=1}^K G_{i,j}^n p_{i,j}$$

□

Corollary 1. The sampling probability $\{p_i^n\}_{i=1,2,\dots,B}^{n \in \mathbb{N}}$ in algorithm 3 has a uniform lower bound π .

Proof. From Assumption 2, for $\forall i \in [1, B], n \in \mathbb{N}^*$, we have

$$\tilde{G}_i^{n,T} = \frac{\sum_{t=1}^T \|g_i^{n,t}\|^2}{T} \leq \frac{T G^2}{T} = G^2$$

We obtain

$$G_i^N = (1 - \beta) G_i^{N-1} + \beta \tilde{G}_i^{N,T} \leq (1 - \beta) G_i^{N-1} + \beta G^2 \quad (7)$$

Let $p = 1 - \beta$, $q = \beta G$.

Updating equation (7) for K times, we obtain

$$G_i^{N+K} \leq \frac{q}{1-p} + p^{N+K-1} \left(G_i^N - \frac{q}{1-p} \right)$$

Noting that $0 < p < 1$, for $\forall k \in \mathbb{N}^*$, we obtain

$$G_i^{N+K} \leq G^2 + (1 - \beta)^N (G_i^N - G^2)$$

Let $\pi_i^* = \max_{1 \leq n \leq N} \{G_i^n, G^2 + (1 - \beta)^N (G_i^N - G^2)\}$

We have

$$\frac{1}{Be^{\eta\pi_i^*}} \leq \frac{\exp(\eta G_i^n)}{\sum_{j=1}^B \exp(\eta G_j^n)} = p_i^n$$

We get the lower bound $\pi_i = \frac{1}{Be^{\eta\pi_i^*}}$

Let $\pi = \min\{\pi_i\}$, which is the uniform lower bound of $\{p_i^n\}_{i=1,2,\dots,B}^{n \in \mathbb{N}}$. □

Lemma 1. Let $\Gamma_{\tau_n}^{n,t} = \text{Diag}^{-1/2}(\tilde{v}_{\tau_n}^{n,t} + \varepsilon, \|v_{\tau_n}^{n,t-1}\|_{\min} + \varepsilon)$, $\Delta\Gamma_{\tau_n}^{n,t} = \Gamma_{\tau_n}^{n,t-1} - \Gamma_{\tau_n}^{n,t}$, $t = 1, 2, \dots, T$

We have the following upper bound estimation

$$\sum_{n=1}^N \sum_{t=1}^T \|\Delta\Gamma_{\tau_n}^{n,t}\| \leq \frac{2}{\sqrt{\varepsilon}}, \quad \sum_{n=1}^N \sum_{t=1}^T \|\Delta\Gamma_{\tau_n}^{n,t}\|^2 \leq \frac{2}{\varepsilon}.$$

Where $\Delta\Gamma_{\tau_n}^{n,0} = \Gamma_{\tau_{n-1}}^{n-1,T} - \Gamma_{\tau_n}^{n,0}$.

Proof. From algorithm 3, we have $v_{\tau_n}^{n,0} = \|\tilde{v}_{\tau_{n-1}}^{n-1,T}\|_{\max} I$, $n \geq 2$.

For notational convenience, we denote Γ_t , $\tilde{v}_t$ as $\Gamma_{\tau_n}^{n,t}$, $\tilde{v}_{\tau_n}^{n,t}$ in this proof.

$$\begin{aligned} \Gamma_t &= \text{Diag}^{-1/2}(\tilde{v}_t + \varepsilon, \|\tilde{v}_t\|_{\min} + \varepsilon) \\ &\preceq \text{Diag}^{-1/2}(\|\tilde{v}_t\|_{\min} + \varepsilon) \\ &\preceq \frac{1}{\sqrt{\|\tilde{v}_{t-1}\|_{\max} + \varepsilon}} I \\ &= \text{Diag}^{-1/2}(\|\tilde{v}_{t-1}\|_{\max} + \varepsilon) \\ &\preceq \text{Diag}^{-1/2}(\tilde{v}_{t-1} + \varepsilon, \|\tilde{v}_{t-1}\|_{\min} + \varepsilon) = \Gamma_{t-1} \end{aligned}$$

which implies that $\Delta\Gamma_t = \Gamma_{t-1} - \Gamma_t$ is positive semi-definite. Hence, $\|\Delta\Gamma_t\| = \lambda_{\max}(\Delta\Gamma_t) \geq 0$

Using the convexity of $\lambda_{\max}$ over symmetric matrices, we have

$$\begin{aligned} \sum_{t=1}^T \|\Delta\Gamma_t\| &= \sum_{t=1}^T \lambda_{\max}(\Gamma_{t-1} - \Gamma_t) \\ &\leq \sum_{t=1}^T (\lambda_{\max}(\Gamma_{t-1}) + \lambda_{\max}(-\Gamma_t)) \\ &= \sum_{t=1}^T (\lambda_{\max}(\Gamma_{t-1}) - \lambda_{\min}(\Gamma_t)) \\ &= \sum_{t=1}^T \left(\frac{1}{\sqrt{\|\tilde{v}_{t-1}\|_{\min} + \varepsilon}} - \frac{1}{\sqrt{\|\tilde{v}_t\|_{\max} + \varepsilon}} \right) \\ &= \sum_{t=1}^T \left(\frac{1}{\sqrt{\|\tilde{v}_{t-1}\|_{\min} + \varepsilon}} - \frac{1}{\sqrt{\|\tilde{v}_{t-1}\|_{\max} + \varepsilon}} + \frac{1}{\sqrt{\|\tilde{v}_{t-1}\|_{\max} + \varepsilon}} - \frac{1}{\sqrt{\|\tilde{v}_t\|_{\max} + \varepsilon}} \right) \\ &\leq \sum_{t=1}^T \left(\frac{1}{\sqrt{\|\tilde{v}_{t-1}\|_{\min} + \varepsilon}} - \frac{1}{\sqrt{\|\tilde{v}_t\|_{\min} + \varepsilon}} \right) + \sum_{t=1}^{NT} \left(\frac{1}{\sqrt{\|\tilde{v}_{t-1}\|_{\max} + \varepsilon}} - \frac{1}{\sqrt{\|\tilde{v}_t\|_{\max} + \varepsilon}} \right) \\ &= \frac{1}{\sqrt{\|\tilde{v}_0\|_{\min} + \varepsilon}} - \frac{1}{\sqrt{\|\tilde{v}_T\|_{\min} + \varepsilon}} + \frac{1}{\sqrt{\|\tilde{v}_0\|_{\max} + \varepsilon}} - \frac{1}{\sqrt{\|\tilde{v}_T\|_{\max} + \varepsilon}} \end{aligned}$$

For the second sum of squared norms, notice that for scalars $a \geq b \geq 0$, it holds that $(a-b)^2 \leq (a+b)(a-b) =$

$a^2 - b^2$. Therefore, the derivation can be repeated without the square roots:

$$\begin{aligned}
\sum_{t=1}^T \|\Delta \Gamma_t\|^2 &= \sum_{t=1}^T (\lambda_{\max}(\Gamma_{t-1} - \Gamma_t))^2 \\
&\leq \sum_{t=1}^T (\lambda_{\max}(\Gamma_{t-1}) + \lambda_{\max}(-\Gamma_t))^2 = \sum_{t=1}^T (\lambda_{\max}(\Gamma_{t-1}) - \lambda_{\min}(\Gamma_t))^2 \\
&\leq \sum_{t=1}^T (\lambda_{\max}(\Gamma_{t-1}))^2 - (\lambda_{\min}(\Gamma_t))^2 \\
&= \sum_{t=1}^T \left(\frac{1}{\|\tilde{v}_{t-1}\|_{\min} + \varepsilon} - \frac{1}{\|\tilde{v}_t\|_{\max} + \varepsilon} \right) \\
&= \sum_{t=1}^T \left(\frac{1}{\|\tilde{v}_{t-1}\|_{\min} + \varepsilon} - \frac{1}{\|\tilde{v}_{t-1}\|_{\max} + \varepsilon} + \frac{1}{\|\tilde{v}_{t-1}\|_{\max} + \varepsilon} - \frac{1}{\|\tilde{v}_t\|_{\max} + \varepsilon} \right) \\
&\leq \sum_{t=1}^T \left(\frac{1}{\|\tilde{v}_{t-1}\|_{\min} + \varepsilon} - \frac{1}{\|\tilde{v}_t\|_{\min} + \varepsilon} \right) + \sum_{t=1}^T \left(\frac{1}{\|\tilde{v}_{t-1}\|_{\max} + \varepsilon} - \frac{1}{\|\tilde{v}_t\|_{\max} + \varepsilon} \right) \\
&= \frac{1}{\|\tilde{v}_0\|_{\min} + \varepsilon} - \frac{1}{\|\tilde{v}_T\|_{\min} + \varepsilon} + \frac{1}{\|\tilde{v}_0\|_{\max} + \varepsilon} - \frac{1}{\|\tilde{v}_T\|_{\max} + \varepsilon}
\end{aligned}$$

Applying the above estimation, we obtain

$$\begin{aligned}
\sum_{n=1}^N \sum_{t=1}^T \|\Delta \Gamma_{\tau_n}^{n,t}\| &\leq \sum_{n=1}^N \left(\frac{1}{\sqrt{\|\tilde{v}_{\tau_n}^{n,0}\|_{\min} + \varepsilon}} - \frac{1}{\sqrt{\|\tilde{v}_{\tau_n}^{n,T}\|_{\min} + \varepsilon}} + \frac{1}{\sqrt{\|\tilde{v}_{\tau_n}^{n,0}\|_{\max} + \varepsilon}} - \frac{1}{\sqrt{\|\tilde{v}_{\tau_n}^{n,T}\|_{\max} + \varepsilon}} \right) \\
&\leq \sum_{n=1}^N \left(\frac{2}{\sqrt{\|\tilde{v}_{\tau_n}^{n,0}\|_{\min} + \varepsilon}} - \frac{2}{\sqrt{\|\tilde{v}_{\tau_n}^{n,T}\|_{\max} + \varepsilon}} \right) \\
&= \frac{2}{\sqrt{\|\tilde{v}_{\tau_1}^{1,0}\|_{\min} + \varepsilon}} - \sum_{n=1}^{N-1} \left(\frac{2}{\sqrt{\|\tilde{v}_{\tau_n}^{n,T}\|_{\max} + \varepsilon}} - \frac{2}{\sqrt{\|\tilde{v}_{\tau_{n+1}}^{n+1,0}\|_{\min} + \varepsilon}} \right) - \frac{2}{\sqrt{\|\tilde{v}_{\tau_N}^{N,T}\|_{\max} + \varepsilon}} \\
&\leq \frac{2}{\sqrt{\|\tilde{v}_{\tau_1}^{1,0}\|_{\min} + \varepsilon}} \leq \frac{2}{\sqrt{\varepsilon}}
\end{aligned}$$

Where the last inequality is because $v_{\tau_{n+1}}^{n+1,0} = \|\tilde{v}_{\tau_n}^{n,T}\|_{\max} I$. Similarly, we obtain

$$\sum_{n=1}^N \sum_{t=1}^T \|\Delta \Gamma_{\tau_n}^{n,t}\|^2 \leq \frac{2}{\varepsilon}.$$

Which completes the proof. $\square$

Lemma 2.

$$\mathbb{E}_t[\|m_{\tau_n}^{n,t}\|] \leq G, \quad \sum_{t=1}^T \mathbb{E}_t \left[\left\| \frac{\beta_1 m_{\tau_n}^{n,t-1}}{1 - \beta_1} \right\|^2 \right] \leq \left(\frac{\beta_1}{1 - \beta_1} \right)^2 G^2 T, \quad \mathbb{E}_t[\|\tilde{v}_{\tau_n}^{n,t}\|_{\max}] \leq G^2$$

Proof.

$$\|m_{\tau_n}^{n,t}\| = \|(1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_{\tau_n}^{n,i}\| \leq (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} G \leq G$$

Taking expectation, we obtain

$$\mathbb{E}_t[\|m_{\tau_n}^{n,t}\|] \leq G \tag{8}$$

From equation (8), we have

$$\left\| \frac{\beta_1 m_{\tau_n}^{n,t-1}}{1-\beta_1} \right\|^2 = \left(\frac{\beta_1}{1-\beta_1} \right)^2 \|m_{\tau_n}^{n,t-1}\|^2 \leq \left(\frac{\beta_1}{1-\beta_1} \right)^2 G^2$$

We get the bound

$$\sum_{t=1}^T \mathbb{E}_t \left[\left\| \frac{\beta_1 m_{\tau_n}^{n,t-1}}{1-\beta_1} \right\|^2 \right] \leq \left(\frac{\beta_1}{1-\beta_1} \right)^2 G^2 T$$

Next, we bound $\|v_{\tau_n}^{n,t}\|$

$$\|v_{\tau_n}^{n,t}\| = \|(1-\beta_2) \sum_{i=1}^t \beta_2^{t-i} (g_{\tau_n}^{n,i})^2\| \leq (1-\beta_2) \sum_{i=1}^t \beta_2^{t-1} G^2 \leq G^2.$$

We obtain

$$\mathbb{E}_t[\|\tilde{v}_{\tau_n}^{n,t}\|_{\max}] \leq G^2.$$

□

Corollary 2. Suppose Assumptions 1, 2, 3 are satisfied. When the learning rate meets the condition that $\alpha \leq \min \left\{ \sqrt{\frac{\varepsilon}{4C_0L}}, \frac{\sqrt{\varepsilon}}{C_1L} \right\}$, inner iterations at n -th block epoch satisfies:

$$\begin{aligned} \mathbb{E}_t[f(x_{\tau_n}^{n,T}) - f(x_{\tau_n}^{n,0})] &\leq -\frac{\alpha}{2C_0} \sum_{t=0}^{T-1} \mathbb{E}_t[\|\nabla_{\tau_n} f(\theta_{\tau_n}^{n,t})\|^2] + \frac{T\alpha^2 L\sigma^2}{\varepsilon} + \frac{T\alpha^3 C_1^2 L^2 C_0 G^2}{\varepsilon^2} \\ &\quad + 2\alpha(1+C_1)G^2 \sum_{t=1}^T \mathbb{E}_t[\|\Delta \Gamma_{\tau_n}^{n,t}\|] + \frac{2\alpha^2 C_1^2 L G^2}{\varepsilon} \end{aligned} \quad (9)$$

Proof. First, we define a series $\{x_{\tau_n}^{n,t}\}$, $t = 0, 1, \dots, T$

$$x_{\tau_n}^{n,t} = \theta_{\tau_n}^{n,t} - \alpha \frac{\beta_1 \Gamma_{\tau_n}^{n,t} m_{\tau_n}^{n,t}}{1-\beta_1}$$

Noting that for $t = 1, 2, \dots, T$, we have

$$m_{\tau_n}^{n,0} = 0, \quad x_{\tau_n}^{n,0} = \theta_{\tau_n}^{n,0}, \quad \theta_{\tau_n}^{n,t} = \theta_{\tau_n}^{n,t-1} - \alpha \Gamma_{\tau_n}^{n,t} m_{\tau_n}^{n,t} \quad (10)$$

We will prove the following recurrence relation

$$x_{\tau_n}^{n,t} = x_{\tau_n}^{n,t-1} - \alpha \Gamma_{\tau_n}^{n,t} g_{\tau_n}^{n,t} + \alpha \Delta \Gamma_{\tau_n}^{n,t} \frac{\beta_1}{1-\beta_1} m_{\tau_n}^{n,t-1}, \quad t = 1, 2, \dots, T \quad (11)$$

For notational convenience, we denote $\theta_t, g_t, m_t, \Gamma_t, \nabla f(\theta_t)$ as $\theta_{\tau_n}^{n,t}, g_{\tau_n}^{n,t}, m_{\tau_n}^{n,t}, \Gamma_{\tau_n}^{n,t}, \nabla f(\theta_{\tau_n}^{n,t})$ below.

From the definition of $\{x_i^{n,t}\}$, we have

$$\begin{aligned} x_t &= \theta_t - \alpha \Gamma_t \frac{\beta_1}{1-\beta_1} m_t \\ &= \theta_{t-1} - \alpha \Gamma_t m_t - \alpha \Gamma_t \frac{\beta_1}{1-\beta_1} m_t \\ &= \theta_{t-1} - \alpha \Gamma_t \frac{1}{1-\beta_1} m_t \\ &= \theta_{t-1} - \alpha \Gamma_t \frac{\beta_1 m_{t-1} + (1-\beta_1) g_t}{1-\beta_1} \\ &= \theta_{t-1} - \alpha \Gamma_t g_t - \alpha \Gamma_t \frac{\beta_1 m_{t-1}}{1-\beta_1} \end{aligned}$$

Noting that

$$x_{t-1} = \theta_{t-1} - \alpha \Gamma_{t-1} \frac{\beta_1}{1 - \beta_1} m_{t-1}$$

We obtain

$$x_t = x_{t-1} - \alpha \Gamma_t g_t + \alpha \Delta \Gamma_t \frac{\beta_1}{1 - \beta_1} m_{t-1}$$

Next, we apply smoothness assumption 1 of the loss function f over the iterates $x_{\tau_n}^{n,t}$, $t = 1, \dots, T$.

$$f(x_t) \leq f(x_{t-1}) + \langle \nabla f(x_{t-1}), x_t - x_{t-1} \rangle + \frac{L}{2} \|x_t - x_{t-1}\|^2.$$

Taking expectation, we obtain

$$\begin{aligned} \mathbb{E}_t[f(x_t)] - \mathbb{E}_t[f(x_{t-1})] &\leq -\alpha \mathbb{E}_t[\langle \nabla f(x_{t-1}), \Gamma_t g_t \rangle] + \alpha \mathbb{E}_t \left[\left\langle \nabla f(x_{t-1}), \Delta \Gamma_t \left(\frac{\beta_1}{1 - \beta_1} m_{t-1} \right) \right\rangle \right] \\ &\quad + \frac{\alpha^2 L}{2} \mathbb{E}_t \left[\left\| \Gamma_t g_t - \Delta \Gamma_t \left(\frac{\beta_1}{1 - \beta_1} m_{t-1} \right) \right\|^2 \right] \end{aligned}$$

Expanding further, we have

$$\begin{aligned} \mathbb{E}_t[f(x_t)] - \mathbb{E}_t[f(x_{t-1})] &\leq -\alpha \mathbb{E}_t[\langle \nabla f(\theta_{t-1}), \Gamma_t g_t \rangle] \quad (\text{I}) \\ &\quad + \alpha \mathbb{E}_t[\langle \nabla f(\theta_{t-1}) - \nabla f(x_{t-1}), \Gamma_t g_t \rangle] \quad (\text{II}) \\ &\quad + \alpha \mathbb{E}_t \left[\left\langle \nabla f(x_{t-1}), \Delta \Gamma_t \left(\frac{\beta_1}{1 - \beta_1} m_{t-1} \right) \right\rangle \right] \quad (\text{III}) \\ &\quad + \frac{\alpha^2 L}{2} \mathbb{E}_t \left[\left\| \Gamma_t g_t - \Delta \Gamma_t \left(\frac{\beta_1}{1 - \beta_1} m_{t-1} \right) \right\|^2 \right] \quad (\text{IV}) \end{aligned}$$

We will estimate each part step by step.

Bounding term I

$$\begin{aligned} I &= -\alpha \mathbb{E}_t[\langle \nabla f(\theta_{t-1}), \Gamma_{t-1} g_t \rangle] + \alpha \mathbb{E}_t[\langle \nabla f(\theta_{t-1}), \Delta \Gamma_t g_t \rangle] \\ &\leq -\alpha \mathbb{E}_t[\langle \nabla f(\theta_{t-1}), \Gamma_{t-1} \nabla f(\theta_{t-1}) \rangle] + \alpha G^2 \mathbb{E}_t[\|\Delta \Gamma_t\|] \\ &\leq -\alpha \lambda_{\min}(\Gamma_{t-1}) \mathbb{E}_t[\|\nabla f(\theta_{t-1})\|^2] + \alpha G^2 \mathbb{E}_t[\|\Delta \Gamma_t\|] \\ &\leq -\frac{\alpha}{C_0} \mathbb{E}_t[\|\nabla f(\theta_{t-1})\|^2] + \alpha G^2 \mathbb{E}_t[\|\Delta \Gamma_t\|] \end{aligned}$$

Where $\frac{1}{C_0} = \frac{1}{\sqrt{G^2 + \varepsilon}} \leq \lambda_{\min}(\Gamma_{t-1})$, the first inequality is because assumption 2, the second inequality is because lemma 2.

Bounding term II

$$\begin{aligned} \text{II} &= \alpha \mathbb{E}_t[\langle \nabla f(\theta_{t-1}) - \nabla f(x_{t-1}), \Gamma_{t-1} g_t \rangle] - \alpha \mathbb{E}_t[\langle \nabla f(\theta_{t-1}) - \nabla f(x_{t-1}), \Delta \Gamma_t g_t \rangle] \\ &\leq \alpha \mathbb{E}_t[\langle \nabla f(\theta_{t-1}) - \nabla f(x_{t-1}), \Gamma_{t-1} \nabla f(\theta_{t-1}) \rangle] + \alpha^2 L \mathbb{E}_t \left[\left\| \Gamma_{t-1} \left(\frac{\beta_1 m_{t-1}}{1 - \beta_1} \right) \right\| \|\Delta \Gamma_t g_t\| \right] \\ &\leq \frac{\alpha \rho}{2\varepsilon} \mathbb{E}_t[\|\nabla f(\theta_{t-1})\|^2] + \frac{\alpha}{2\rho} \mathbb{E}_t[\|\nabla f(\theta_{t-1}) - \nabla f(x_{t-1})\|^2] + \frac{\alpha^2 C_1 L G^2}{\sqrt{\varepsilon}} \mathbb{E}_t[\|\Delta \Gamma_t\|] \\ &\leq \frac{\alpha \rho}{2\varepsilon} \mathbb{E}_t[\|\nabla f(\theta_{t-1})\|^2] + \frac{\alpha^3 L^2}{2\rho} \mathbb{E}_t \left[\left\| \Gamma_{t-1} \left(\frac{\beta_1 m_{t-1}}{1 - \beta_1} \right) \right\|^2 \right] + \frac{\alpha^2 C_1 L G^2}{\sqrt{\varepsilon}} \mathbb{E}_t[\|\Delta \Gamma_t\|] \\ &\leq \frac{\alpha \rho}{2\varepsilon} \mathbb{E}_t[\|\nabla f(\theta_{t-1})\|^2] + \frac{\alpha^3 C_1^2 L^2 G^2}{2\rho \varepsilon} + \frac{\alpha^2 C_1 L G^2}{\sqrt{\varepsilon}} \mathbb{E}_t[\|\Delta \Gamma_t\|] \end{aligned}$$

where $C_1 = \frac{\beta_1}{1-\beta_1}$, ρ is a positive constant, we used the fact $\|\Gamma_t\| = \frac{1}{\sqrt{\|\tilde{v}_t\|_{\min} + \varepsilon}} \leq \frac{1}{\sqrt{\varepsilon}}$, the first and third inequality is because assumption 1, the second inequality is because Cauchy–Schwarz inequality.

Bounding term III

$$\begin{aligned}
\text{III} &= \alpha \mathbb{E}_t \left[\left\langle \nabla f(x_{t-1}), \Delta \Gamma_t \left(\frac{\beta_1}{1-\beta_1} m_{t-1} \right) \right\rangle \right] \\
&\leq \alpha \mathbb{E}_t \left[\left\langle \nabla f(\theta_{t-1}), \Delta \Gamma_t \left(\frac{\beta_1}{1-\beta_1} m_{t-1} \right) \right\rangle \right] + \alpha \mathbb{E}_t \left[\left\langle \nabla f(x_{t-1}) - \nabla f(\theta_{t-1}), \Delta \Gamma_t \left(\frac{\beta_1}{1-\beta_1} m_{t-1} \right) \right\rangle \right] \\
&\leq \alpha \mathbb{E}_t \left[\|\nabla f(\theta_{t-1})\| \|\Delta \Gamma_t \frac{\beta_1 m_{t-1}}{1-\beta_1}\| \right] + \alpha^2 L \mathbb{E}_t \left[\left\| \Gamma_{t-1} \left(\frac{\beta_1 m_{t-1}}{1-\beta_1} \right) \right\| \|\Delta \Gamma_t \frac{\beta_1 m_{t-1}}{1-\beta_1}\| \right] \\
&\leq \alpha C_1 G^2 \mathbb{E}_t [\|\Delta \Gamma_t\|] + \frac{\alpha^2 C_1^2 L G^2}{\sqrt{\varepsilon}} \mathbb{E}_t [\|\Delta \Gamma_t\|]
\end{aligned}$$

Where the third inequality is because assumption 1.

Bounding term IV

$$\begin{aligned}
\text{IV} &= \frac{\alpha^2 L}{2} \mathbb{E}_t \left[\left\| \Gamma_t g_t - \Delta \Gamma_t \left(\frac{\beta_1}{1-\beta_1} m_{t-1} \right) \right\|^2 \right] \\
&\leq \alpha^2 L \mathbb{E}_t [\|\Gamma_t g_t\|^2] + \alpha^2 L \mathbb{E}_t \left[\left\| \Delta \Gamma_t \left(\frac{\beta_1}{1-\beta_1} m_{t-1} \right) \right\|^2 \right] \\
&\leq \frac{\alpha^2 L}{\varepsilon} \mathbb{E}_t [\|g_t - \nabla f(\theta_{t-1}) + \nabla f(\theta_{t-1})\|^2] + \alpha^2 L \mathbb{E}_t \left[\left\| \Delta \Gamma_t \left(\frac{\beta_1 m_{t-1}}{1-\beta_1} \right) \right\|^2 \right] \\
&\leq \frac{\alpha^2 L}{\varepsilon} (\mathbb{E}_t [\|\nabla f(\theta_{t-1})\|^2] + \sigma^2) + \alpha^2 C_1^2 L G^2 \mathbb{E}_t [\|\Delta \Gamma_t\|^2] \\
&\leq \frac{\alpha^2 L}{\varepsilon} \mathbb{E}_t [\|\nabla f(\theta_{t-1})\|^2] + \frac{\alpha^2 L \sigma^2}{\varepsilon} + \alpha^2 C_1^2 L G^2 \mathbb{E}_t [\|\Delta \Gamma_t\|^2]
\end{aligned}$$

Where we used assumption 3 that g_t is unbiased with bounded variance σ^2 .

We obtain the bound of each part

$$\begin{aligned}
\text{I} &\leq -\frac{\alpha}{C_0} \mathbb{E}_t [\|\nabla f(\theta_{t-1})\|^2] + \alpha G^2 \mathbb{E}_t [\|\Delta \Gamma_t\|] \\
\text{II} &\leq \frac{\alpha \rho}{2\varepsilon} \mathbb{E}_t [\|\nabla f(\theta_{t-1})\|^2] + \frac{\alpha^3 C_1^2 L^2 G^2}{2\rho\varepsilon} + \frac{\alpha^2 C_1 L G^2}{\sqrt{\varepsilon}} \mathbb{E}_t [\|\Delta \Gamma_t\|] \\
\text{III} &\leq \alpha C_1 G^2 \mathbb{E}_t [\|\Delta \Gamma_t\|] + \frac{\alpha^2 C_1^2 L G^2}{\sqrt{\varepsilon}} \mathbb{E}_t [\|\Delta \Gamma_t\|] \\
\text{IV} &\leq \frac{\alpha^2 L}{\varepsilon} \mathbb{E}_t [\|\nabla f(\theta_{t-1})\|^2] + \frac{\alpha^2 L \sigma^2}{\varepsilon} + \alpha^2 C_1^2 L G^2 \mathbb{E}_t [\|\Delta \Gamma_t\|^2]
\end{aligned}$$

Substituting each part into the above single-step estimation, we obtain

$$\begin{aligned}
\mathbb{E}_t[f(x_t) - f(x_{t-1})] &\leq \left(-\frac{\alpha}{C_0} + \frac{\alpha^2 L}{\varepsilon} + \frac{\alpha \rho}{2\varepsilon} \right) \mathbb{E}_t [\|\nabla f(\theta_{t-1})\|^2] + \frac{\alpha^2 L \sigma^2}{\varepsilon} + \frac{\alpha^3 C_1^2 L^2 G^2}{2\rho\varepsilon} \\
&\quad + \left(\alpha(1+C_1)G^2 + \frac{\alpha^2(1+C_1)C_1 L G^2}{\sqrt{\varepsilon}} \right) \mathbb{E}_t [\|\Delta \Gamma_t\|] + \alpha^2 C_1^2 L G^2 \mathbb{E}_t [\|\Delta \Gamma_t\|^2]
\end{aligned}$$

Updating T times, we obtain

$$\mathbb{E}_t[f(x_T) - f(x_0)] \leq \left(-\frac{\alpha}{C_0} + \frac{\alpha^2 L}{\varepsilon} + \frac{\alpha \rho}{2\varepsilon} \right) \sum_{t=0}^{T-1} \mathbb{E}_t [\|\nabla f(\theta_t)\|^2] + \frac{T \alpha^2 L \sigma^2}{\varepsilon} + \frac{T \alpha^3 C_1^2 L^2 G^2}{2\rho\varepsilon}$$

$$+ \left(\alpha(1 + C_1)G^2 + \frac{\alpha^2(1 + C_1)C_1LG^2}{\sqrt{\varepsilon}} \right) \sum_{t=1}^T \mathbb{E}_t [\|\Delta \Gamma_t\|] + \alpha^2 C_1^2 LG^2 \sum_{t=1}^T \mathbb{E}_t [\|\Delta \Gamma_t\|^2].$$

Choosing $\rho = \frac{\varepsilon}{2C_0}$, $\alpha \leq \min \left\{ \sqrt{\frac{\varepsilon}{4C_0L}}, \frac{\sqrt{\varepsilon}}{C_1L} \right\}$, applying lemma 1, we obtain:

$$\begin{aligned} \mathbb{E}_t[f(x_T) - f(x_0)] &\leq -\frac{\alpha}{2C_0} \sum_{t=0}^{T-1} \mathbb{E}_t[\|\nabla f(\theta_t)\|^2] + \frac{T\alpha^2 L\sigma^2}{\varepsilon} + \frac{T\alpha^3 C_1^2 L^2 C_0 G^2}{\varepsilon^2} \\ &\quad + 2\alpha(1 + C_1)G^2 \sum_{t=1}^T \mathbb{E}_t[\|\Delta \Gamma_t\|] + \frac{2\alpha^2 C_1^2 LG^2}{\varepsilon} \end{aligned}$$

□

Corollary 3. For each inner iteration index $T_1 = 0, 1, \dots, T$ at n -th block epoch, the expectation of $\|\nabla_{\tau_n} f(\theta^{n,0})\|^2$ is bounded by $\|\nabla_{\tau_n} f(\theta_{\tau_n}^{n,T_1})\|^2$

$$\frac{\mathbb{E}_t[\|\nabla_{\tau_n} f(\theta^{n,0})\|^2] - \alpha^2 T^2 C_2}{2} \leq \mathbb{E}_t[\|\nabla_{\tau_n} f(\theta_{\tau_n}^{n,T_1})\|^2] \quad (12)$$

Where $C_2 = \frac{2L^2 G^2}{\varepsilon}$.

Proof.

$$\begin{aligned} \mathbb{E}_t[\|\nabla_{\tau_n} f(\theta^{n,0})\|^2] &\leq 2\mathbb{E}_t[\|\nabla_{\tau_n} f(\theta_{\tau_n}^{n,T_1})\|^2] + 2\mathbb{E}_t[\|\nabla_{\tau_n} f(\theta^{n,0}) - \nabla_{\tau_n} f(\theta_{\tau_n}^{n,T_1})\|^2] \\ &\leq 2\mathbb{E}_t[\|\nabla_{\tau_n} f(\theta_{\tau_n}^{n,T_1})\|^2] + 2\mathbb{E}_t[\|\nabla f(\theta^{n,0}) - \nabla f(\bar{\theta}_{\tau_n}^{n,T_1})\|^2] \\ &\leq 2\mathbb{E}_t[\|\nabla_{\tau_n} f(\theta_{\tau_n}^{n,T_1})\|^2] + 2L^2 \mathbb{E}_t[\|\theta^{n,0} - \bar{\theta}_{\tau_n}^{n,T_1}\|^2] \end{aligned}$$

Where

$$\begin{aligned} \bar{\theta}_{\tau_n}^{n,T_1} &= (\theta_1^{n-1,T}, \dots, \theta_{\tau_n-1}^{n-1,T}, \theta_{\tau_n}^{n,T_1}, \theta_{\tau_n+1}^{n-1,T}, \dots, \theta_B^{n-1,T}) \\ \theta^{n,0} &= (\theta_1^{n-1,T}, \dots, \theta_{\tau_n-1}^{n-1,T}, \theta_{\tau_n}^{n,0}, \theta_{\tau_n+1}^{n-1,T}, \dots, \theta_B^{n-1,T}), \end{aligned}$$

the second inequality is because the partial derivative vector $\nabla_{\tau_n} f(\theta_{\tau_n}^{n,T_1})$ is part of the gradient vector $\nabla f(\bar{\theta}_{\tau_n}^{n,T_1})$.

Next, we will bound $\mathbb{E}_t[\|\theta^{n,0} - \bar{\theta}_{\tau_n}^{n,T_1}\|^2]$.

$$\begin{aligned} \mathbb{E}_t[\|\theta^{n,0} - \bar{\theta}_{\tau_n}^{n,T_1}\|^2] &= \alpha^2 \mathbb{E}_t \left[\left\| \sum_{j=1}^{T_1} \Gamma_{\tau_n}^{n,j} m_{\tau_n}^{n,j} \right\|^2 \right] \\ &\leq \frac{T_1 \alpha^2}{\varepsilon} \sum_{j=1}^{T_1} \mathbb{E}_t[\|m_{\tau_n}^{n,j}\|^2] \\ &\leq \frac{T_1^2 \alpha^2 G^2}{\varepsilon} \leq \frac{T^2 \alpha^2 G^2}{\varepsilon} \end{aligned}$$

Where we used assumption 3 and lemma 2, the first inequality is because Cauchy-shwarz inequality.

We obtain

$$\mathbb{E}_t[\|\nabla_{\tau_n} f(\theta^{n,0})\|^2] \leq 2\mathbb{E}_t[\|\nabla_{\tau_n} f(\theta_{\tau_n}^{n,T_1})\|^2] + \frac{2T^2 \alpha^2 L^2 G^2}{\varepsilon}.$$

Let $C_2 = \frac{2L^2 G^2}{\varepsilon}$, we have

$$\frac{\mathbb{E}_t[\|\nabla_{\tau_n} f(\theta^{n,0})\|^2] - \alpha^2 T^2 C_2}{2} \leq \mathbb{E}_t[\|\nabla_{\tau_n} f(\theta_{\tau_n}^{n,T_1})\|^2].$$

□

D.2 Proof of theorem 1

Theorem 2. (*MISA Convergence*) Suppose that Assumptions 1, 2, 3 are fulfilled, learning rate satisfies $\alpha \leq \min \left\{ \sqrt{\frac{\varepsilon}{4C_0L}}, \frac{\sqrt{\varepsilon}}{C_1L} \right\}$, MISA converges at the rate of $\mathcal{O}(\frac{1}{\sqrt{NT}} + \frac{1}{N})$.

$$\frac{\sum_{n=1}^N \mathbb{E}_t[\|\nabla f(\theta^{n,0})\|^2]}{N} \leq \frac{1}{\sqrt{N}} \frac{4C_0 \sqrt{\Delta_0 [TL\sigma^2 + 2LG^2(\frac{\beta_1}{1-\beta_1})^2]}}{T\pi\sqrt{\varepsilon}} + \mathcal{O}(\frac{1}{N}).$$

Where $\Delta_0 = f(x^{1,0}) - f(x^*)$, $C_1 = \frac{\beta_1}{1-\beta_1}$, $C_0 = \frac{1}{\sqrt{G^2+\varepsilon}}$.

Proof. From algorithm 3, we have

$$\theta_{\tau_{n+1}}^{n+1,0} = \theta_{\tau_n}^{n,T} - \alpha \frac{\beta_1}{1-\beta_1} \Gamma_{\tau_n}^{n,T} m_{\tau_n}^{n,T}$$

From the definition of the sequence $\{x_{\tau_n}^{n,T}\}$ in (10), we obtain

$$\mathbb{E}_t[f(x_{\tau_n}^{n+1,0})] = \mathbb{E}_t[f(\theta_{\tau_n}^{n+1,0})] = \mathbb{E}_t[f(x_{\tau_n}^{n,T})]$$

Which implies

$$\mathbb{E}_t[f(x_{\tau_n}^{n+1,0})] = \mathbb{E}_t[f(x_{\tau_n}^{n,T})] \quad (13)$$

Substituting (12) and (13) into (9), we obtain

$$\begin{aligned} \mathbb{E}_t[f(x_{\tau_n}^{n+1,0}) - f(x_{\tau_n}^{n,0})] &\leq -\frac{\alpha T}{4C_0} \mathbb{E}_t[\|\nabla_{\tau_n} f(\theta^{n,T})\|^2] + \frac{T\alpha^2 L\sigma^2}{\varepsilon} + \frac{T\alpha^3 C_1^2 L^2 C_0 G^2}{\varepsilon^2} \\ &\quad + 2\alpha(1+C_1)G^2 \sum_{t=1}^T \mathbb{E}_t[\|\Delta \Gamma_t\|] + \frac{2\alpha^2 C_1^2 LG^2}{\varepsilon} + \frac{\alpha^3 T^3 C_2}{4C_0} \end{aligned}$$

It's obvious that the norm of full gradient is equal to the sum of individual block's gradient norms.

$$\|\nabla f(\theta^{n,T})\|^2 = \sum_{b=1}^B \|\nabla_b f(\theta^{n,T})\|^2$$

Taking expectation, we have

$$\mathbb{E}_t[\|\nabla f(\theta^{n,T})\|^2] = \sum_{b=1}^B \mathbb{E}_t[\|\nabla_b f(\theta^{n,T})\|^2]$$

Using the conclusion of Corollary 2 and taking expectation with respect to τ_n , we have

$$\begin{aligned} \mathbb{E}_{t,\tau_n}[f(x_{\tau_n}^{n+1,0}) - f(x_{\tau_n}^{n,0})] &= \sum_{b=1}^B p_b^n \mathbb{E}_t[f(x_b^{n+1,0}) - f(x_b^{n,0})] \\ &\leq -\frac{\alpha T}{4C_0} \sum_{b=1}^B P_b^n \mathbb{E}_t[\|\nabla_b f(\theta^{n,0})\|^2] + \frac{T\alpha^2 L\sigma^2}{\varepsilon} + \frac{T\alpha^3 C_1^2 L^2 C_0 G^2}{\varepsilon^2} \\ &\quad + 2\alpha(1+C_1)G^2 \sum_{b=1}^B P_b^n \left(\sum_{t=1}^T \mathbb{E}_t[\|\Delta \Gamma_b^{n,t}\|] \right) + \frac{2\alpha^2 C_1^2 LG^2}{\varepsilon} + \frac{\alpha^3 T^3 C_2}{4C_0} \\ &\leq -\frac{\alpha T}{4C_0} \pi \mathbb{E}_t[\|\nabla f(\theta^{n,0})\|^2] + \frac{T\alpha^2 L\sigma^2}{\varepsilon} + \frac{T\alpha^3 C_1^2 L^2 C_0 G^2}{\varepsilon^2} \\ &\quad + 2\alpha(1+C_1)G^2 \sum_{b=1}^B P_b^n \left(\sum_{t=1}^T \mathbb{E}_t[\|\Delta \Gamma_b^{n,t}\|] \right) + \frac{2\alpha^2 C_1^2 LG^2}{\varepsilon} + \frac{\alpha^3 T^3 C_2}{4C_0} \end{aligned}$$

$$\begin{aligned} &\leq -\frac{\alpha T}{4C_0} \pi \mathbb{E}_t [\|\nabla f(\theta^{n,0})\|^2] + \frac{T\alpha^2 L\sigma^2}{\varepsilon} + \frac{T\alpha^3 C_1^2 L^2 C_0 G^2}{\varepsilon^2} \\ &\quad + 2\alpha(1+C_1)G^2 \sum_{t=1}^T \mathbb{E}_t [\|\Delta \Gamma_{b_n}^{n,t}\|] + \frac{2\alpha^2 C_1^2 L G^2}{\varepsilon} + \frac{\alpha^3 T^3 C_2}{4C_0} \end{aligned}$$

Where $b_n = \arg \max_{b \in \{1,2,\dots,B\}} \left\{ \sum_{t=1}^T \mathbb{E}_t [\|\Delta \Gamma_b^{n,t}\|] \right\}$, $\{P_b^n\}$ is the sampling probability of each block at n -th block epoch, the first inequality is because Corollary 2, the second inequality is because Corollary 1.

Let $D_0 = \frac{\pi}{4C_0}$, $D_1 = \frac{L\sigma^2}{\varepsilon}$, $D_2 = \frac{C_1^2 L^2 C_0 G^2}{\varepsilon^2}$, $D_3 = 2(1+C_1)G^2$, $D_4 = \frac{2C_1^2 L G^2}{\varepsilon}$, $D_5 = \frac{C_2}{4C_0}$, we obtain

$$\begin{aligned} \mathbb{E}_t[f(x^{n+1,0}) - f(x^{n,0})] &\leq -\alpha T D_0 \mathbb{E}_t [\|\nabla f(\theta^{n,0})\|^2] + T\alpha^2 D_1 + T\alpha^3 D_2 + \alpha D_3 \sum_{t=1}^T \mathbb{E}_t [\|\Delta \Gamma_{b_n}^{n,t}\|] \\ &\quad + D_4 \alpha^2 + D_5 \alpha^3 T^3 \end{aligned}$$

Updating the above equation for N times, we obtain

$$\begin{aligned} \mathbb{E}_t[f(x^{N+1,0}) - f(x^{1,0})] &\leq -\alpha T D_0 \sum_{n=1}^N \mathbb{E}_t [\|\nabla f(\theta^{n,0})\|^2] + N T \alpha^2 D_1 + N T \alpha^3 D_2 \\ &\quad + \alpha D_3 \sum_{n=1}^N \sum_{t=1}^T \mathbb{E}_t [\|\Delta \Gamma_{b_n}^{n,t}\|] + N D_4 \alpha^2 + N D_5 \alpha^3 T^3 \end{aligned}$$

From lemma 1, we know

$$\sum_{n=1}^N \sum_{t=1}^T \mathbb{E}_t [\|\Delta \Gamma_{b_n}^{n,t}\|] \leq \frac{2}{\sqrt{\varepsilon}}$$

Let $D_6 = -(f(x^*) - f(x^{1,0}))$, we obtain

$$\begin{aligned} \frac{\sum_{n=1}^N \mathbb{E}_t [\|\nabla f(\theta^{n,0})\|^2]}{N} &\leq \frac{D_6}{\alpha N T D_0} + \frac{\alpha D_1}{D_0} + \frac{\alpha^2 D_2}{D_0} + \frac{2D_3}{N T D_0 \sqrt{\varepsilon}} + \frac{\alpha D_4}{T D_0} + \frac{T^2 \alpha^2 D_5}{D_0} \\ &= \frac{D_6}{\alpha N T D_0} + \alpha \frac{T D_1 + D_4}{T D_0} + \alpha^2 \frac{(D_2 + T^2 D_5)}{D_0} \end{aligned}$$

Choosing $\alpha = \sqrt{\frac{D_6}{N(TD_1 + D_4)}}$, we get the conclusion

$$\begin{aligned} \frac{\sum_{n=1}^N \mathbb{E}_t [\|\nabla f(\theta^{n,0})\|^2]}{N} &\leq \frac{2\sqrt{D_6(TD_1 + D_4)}}{\sqrt{N} T D_0} + \mathcal{O}\left(\frac{1}{N}\right) \\ &= \frac{1}{\sqrt{N}} \frac{4C_0 \sqrt{\Delta_0 [TL\sigma^2 + 2LG^2(\frac{\beta_1}{1-\beta_1})^2]}}{T\pi\sqrt{\varepsilon}} + \mathcal{O}\left(\frac{1}{N}\right). \end{aligned}$$

□

E Memory Analysis

The memory analysis presented in this study is conducted under frozen embedding layer and language modeling head layer configurations, with memory quantification comprehensively defined as the cumulative composition of parameters, gradients, optimizer states, and intermediate activation.

Notation. We suppose that the model follows the standard transformer architecture, where each layer consists of six modules, i.e., $W_Q \in \mathbb{R}^{h \times h}$, $W_K \in \mathbb{R}^{h \times h}$, $W_V \in \mathbb{R}^{h \times h}$, $W_O \in \mathbb{R}^{h \times h}$, $W_1 \in \mathbb{R}^{h \times 4h}$, $W_2 \in \mathbb{R}^{4h \times h}$. Let L denote the number of the transformer layers, and a denotes the number of attention heads. v is the vocabulary size of the model. s refer to the sequence length. b represents the training batch size. The

embedding representation dim is h .

Table 13: **Parameter dimension**

Parameter	Property	Memory consumption
X	Activation	$s * h * b = bsh$
W_Q	Model parameter	$a * (h * h/a) = h^2$
Q	Activation	$a * (s * h/a) * b = bsh$
W_K	Model parameter	$a * (h * h/a) = h^2$
K	Activation	$s * (h * h/a) * b = bsh$
W_V	Model parameter	$a * (h * h/a) = h^2$
V	Activation	$a * (s * h/a) * b = bsh$
S	Activation	$a * s * s * b = abs^2$
O	Activation	$a * (s * h/a) * b = bsh$
W_O	Model parameter	$a * (h * h/a) = h^2$
U	Activation	$s * h * b = bsh$
Z	Activation	$s * h * b = bsh$
Z_1	Activation	$s * 4h * b = 4bsh$
W_1	Model parameter	$h * 4h = 4h^2$
$\mathbb{I}_{ZW_1+b_1>0}$	Activation	$s * 4h * b = 4bsh$
W_2	Model parameter	$4h * h = 4h^2$

E.1 Memory analysis of layer-wise optimization method

In this section, we will give detailed memory analysis for layer-wise optimization methods in backward propagation based on LLaMA’s standard architecture.

Lemma 3. $\frac{\partial f}{\partial U} = g_1\left(\frac{\partial f}{\partial Z}, U\right), \quad \frac{\partial f}{\partial A} = g_2\left(\frac{\partial f}{\partial S}, S\right)$

Where $Z = \text{LayerNorm}(U)$, $S = \text{softmax}(A)$.

Proof. First, we compute $\frac{\partial f}{\partial U}$.

The computational flow of $Z = \text{LayerNorm}(U)$ can be written as

$$\begin{aligned}\mu &= \frac{1}{h} \sum_{i=1}^h U_{:,i} \\ \sigma &= \sqrt{\frac{1}{h} \sum_{i=1}^h (U_{:,i} - \mu)^2 + \delta} \\ Z &= \frac{U - \mu}{\sigma}\end{aligned}$$

Taking the partial derivative of U , we obtain

$$\frac{\partial f}{\partial U_{ij}} = \frac{1}{\sigma_j} \left(\frac{\partial f}{\partial Z_{ij}} - \frac{1}{h} \sum_{k=1}^h \left\{ \frac{\partial f}{\partial Z_{ik}} - \frac{U_{ij} - \sigma_j}{\sigma_j^2} \frac{\partial f}{\partial Z_{ik}} (U_{ik} - \mu_j) \right\} \right)$$

We observe that the computation of $\frac{\partial f}{\partial U}$ depends solely on $\frac{\partial f}{\partial Z}$ and U . For notational convenience, we denote $\frac{\partial f}{\partial U}$ as $g_1\left(\frac{\partial f}{\partial Z}, U\right)$.

Second, we compute $\frac{\partial f}{\partial A}$.

Let $S_{ij} = \frac{e^{A_{ij}}}{\sum_{k=1}^s e^{A_{ik}}}$, we have

$$\frac{\partial S_{ij}}{\partial A_{mn}} = \begin{cases} 0, & \text{if } m \neq i \\ -\frac{e^{A_{im}} e^{A_{ij}}}{\left(\sum_{k=1}^s e^{A_{ik}}\right)^2} = -S_{in} S_{ij}, & \text{if } m = i \text{ and } n \neq j \\ \frac{e^{A_{ij}} \left(\sum_{k \neq j}^s e^{A_{ik}}\right)}{\left(\sum_{k=1}^s e^{A_{ik}}\right)^2} = S_{ij} (1 - S_{ij}), & \text{if } m = i \text{ and } n = j \end{cases}$$

We obtain

$$\begin{aligned} \frac{\partial f}{\partial A_{ij}} &= \sum_{k \neq j} \frac{\partial f}{\partial S_{ik}} (-S_{ik} S_{ij}) + \frac{\partial f}{\partial S_{ij}} S_{ij} (1 - S_{ij}) \\ &= S_{ij} \left(\frac{\partial f}{\partial S_{ij}} - \sum_{k=1}^s \frac{\partial f}{\partial S_{ik}} S_{ik} \right) \end{aligned}$$

Which indicates

$$\frac{\partial f}{\partial A} = g_2 \left(\frac{\partial f}{\partial S}, S \right).$$

□

Below, we present the forward and backward propagation flows of certain transformer layer.

	$Q = XW_Q$	$\frac{\partial f}{\partial W_Q} = X^T \frac{\partial f}{\partial Q}$	
	$K = XW_K$	$\frac{\partial f}{\partial W_K} = X^T \frac{\partial f}{\partial K}$	
	$V = XW_V$	$\frac{\partial f}{\partial X} = \frac{\partial f}{\partial V} W_V^T, \frac{\partial f}{\partial W_V} = X^T \frac{\partial f}{\partial V}$	
	$A = \frac{QK^T}{\sqrt{d_k}}$	$\frac{\partial f}{\partial Q} = \frac{\partial f}{\partial A} K^T \frac{1}{\sqrt{d_k}}, \frac{\partial f}{\partial K} = \left(\frac{\partial f}{\partial A}\right)^T Q \frac{1}{\sqrt{d_k}}$	
	$S = \text{softmax}(A)$	$\frac{\partial f}{\partial A} = g_2 \left(\frac{\partial f}{\partial S}, S \right)$	
Forward	$O = SV$	$\frac{\partial f}{\partial S} = \frac{\partial f}{\partial O} V^T, \frac{\partial f}{\partial V} = S^T \frac{\partial f}{\partial O}$	Backward
	$\text{Attn} = OW_0$	$\frac{\partial f}{\partial O} = \frac{\partial f}{\partial \text{Attn}} W_0^T, \frac{\partial f}{\partial W_0} = O^T \frac{\partial f}{\partial \text{Attn}}$	
	$U = \text{Attn} + X$	$\frac{\partial f}{\partial \text{Attn}} = \frac{\partial f}{\partial U}$	
	$Z = \text{LayerNorm}(U)$	$\frac{\partial f}{\partial U} = g_1 \left(\frac{\partial f}{\partial Z}, U \right)$	
	$Z_1 = \text{ReLU}(ZW_1 + b_1)$	$\frac{\partial f}{\partial Z} = \frac{\partial f}{\partial Z_1} \odot \mathbb{I}_{ZW_1 + b_1 > 0} W_1^T, \frac{\partial f}{\partial W_1} = Z^T \frac{\partial f}{\partial Z_1}$	
	$Z_2 = Z_1 W_2 + b_2$	$\frac{\partial f}{\partial Z_1} = \frac{\partial f}{\partial Z_2} W_2^T, \frac{\partial f}{\partial W_2} = Z_1^T \frac{\partial f}{\partial Z_2}$	

Next, we will analyze layer-wise optimization methods' memory based on the forward and backward propagation flow of a transformer layer.

Activation memory of frozen layers. To enable backpropagation, the activation gradients of a Transformer layer must be stored, including the tensors Q, V, K, S, U and $\mathbb{I}_{ZW_1 + b_1 > 0}$. The total activation memory is calculated as: $3bsh + abs^2 + 5bsh = abs^2 + 8bsh$.

Activation memory of activated layers. In the layer-wise optimization method, the weight matrices of unfrozen layers $W_Q, W_K, W_V, W_O, W_1, W_2$ require computation, necessitating the storage of $X, Q, K, V, O, S, U, \mathbb{I}_{ZW_1 + b_1 > 0}, Z$ and Z_1 . The total activation memory here is: $5bsh + abs^2 + 10bsh = abs^2 + 15bsh$.

Parameter memory of layer-wise optimization method. Irrespective of which Transformer layer is activated, the model parameters must always be stored. This includes the weight matrices of a full

Transformer layer $W_Q, W_K, W_V, W_O, W_1, W_2$. The total parameter memory is: $L(4h^2 + 8h^2) = 12h^2L$.

Optimizer state and gradient memory of frozen layer. For frozen layers, neither optimizer states nor gradients need to be stored, resulting in zero memory usage for these components.

Optimizer state and gradient memory of activated layer. Activated layers require storing both gradients and optimizer states. The total memory for this is $3 \times 12h^2 = 36h^2$.

Table 14: **Memory of layer-wise optimization method**

Layer State	Parameter Memory	Activation Memory	Optimizer State and Gradient Memory
Activated layer	$12h^2L$	$abs^2 + 15bsh$	$36h^2$
Frozen layer	$12h^2L$	$abs^2 + 8bsh$	0

In practice, we assume that the activated layer’s ID is i , we obtain the following conclusion

$$\begin{aligned}
& \text{Parameter} : 12h^2L \\
& \text{Optimizer State} : 24h^2 \\
& \text{Gradient} : 12h^2 \\
& \text{Activation} : (L - i)(abs^2 + 8bsh) + (abs^2 + 15bsh) \\
& \text{Total} : (L - i)(abs^2 + 8bsh) + (abs^2 + 15bsh) + 12h^2L + 36h^2
\end{aligned}$$

The rationale for the activation memory is as follows: when the activated layer ID is i , backpropagation only needs to propagate to the i -th layer. Consequently, the activations of layers $j = 1, 2, \dots, i - 1$ can be saved.

Based on this, we derive the peak memory of the layer-wise method.

$$L(abs^2 + 8bsh) + 7bsh + 12h^2L + 36h^2.$$

E.2 Memory analysis of MISA and LoRA

In this section, we focus on the scenario where only a specific module within a layer is activated at each step. Based on this, we derive a memory comparison between the module-wise optimization method and LoRA.

E.2.1 Memory analysis of Module-wise BCD

Based on the computation flow described above, activating different individual modules incurs additional memory consumption compared to the module-wise optimization method.

Table 15: **Memory of Module-wise BCD**

Unfrozen Module	Extra Storage	Extra Activation Memory	Extra Optimizer State And Gradient Memory
W_Q	X, W_Q	bsh	$3h^2$
W_K	X, W_K	bsh	$3h^2$
W_V	X, W_V	bsh	$3h^2$
W_O	O, W_O	bsh	$3h^2$
W_1	Z, W_1	bsh	$12h^2$
W_2	Z_1, W_2	$4bsh$	$12h^2$

Let us consider the case where the activated module is W_Q as an example. The memory analysis yields the following components:

- **Activation memory of frozen modules**

From the layer-wise memory analysis, we know that for a frozen layer, the activation memory requirement is $abs^2 + 8bsh$.

- **Activation memory of the activated module**

To enable backpropagation through W_Q , we need to store the additional activation X , resulting in a total memory of $abs^2 + 9bsh$.

- **Parameter memory in layer-wise optimization**

The layer-wise approach requires storing all parameters of each layer, amounting to $12h^2L$ memory.

- **Optimizer state and gradient memory for frozen layers**

Similar to the layer-wise optimization case, the module-wise approach requires no additional memory for frozen layers.

- **Optimizer state and gradient memory for the activated layer**

Only the optimizer states and gradients for W_Q need to be stored, requiring $3h^2$ memory.

The peak memory consumption for the module-wise optimization method, when W_Q is the activated module, is therefore:

$$L(abs^2 + 8bsh) + bsh + 12h^2L + 3h^2$$

E.2.2 Memory analysis of LoRA

Consider a weight matrix W_Q with LoRA adaptation, which can be expressed as:

$$W_Q = W_0 + B_Q A_Q, \quad \text{where } B_Q \in \mathbb{R}^{h \times r}, A_Q \in \mathbb{R}^{r \times \frac{h}{a}}$$

The backward propagation gradients are computed as:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial B_Q} &= \frac{\partial \mathcal{L}}{\partial W_Q} A_Q^\top \\ \frac{\partial \mathcal{L}}{\partial A_Q} &= B_Q^\top \frac{\partial \mathcal{L}}{\partial W_Q} \end{aligned}$$

This formulation leads to the following memory requirements:

- **Embedding layer activations:** Requires storing parameters of size sh .
- **Adapter gradients and optimizer states:** For each layer's adapters A_Q and B_Q , this requires $3aL \left(\frac{2hr}{a} \right) = 6hrL$ memory.
- **Full model parameters:** The base model parameters require $12h^2L$ memory.
- **Activation storage:** During backpropagation through W_Q , LoRA needs to store additional activations X , A_Q , and B_Q , requiring $L(abs^2 + 9bsh + 2hr)$ memory.

The peak memory consumption when applying LoRA to W_Q is therefore:

$$L(abs^2 + 9bsh + 12h^2 + 8hr)$$

From the analysis above, we can similarly generalize the memory storage of different modules.

E.3 Memory analysis of GaLore

Consider activating W_Q as an example, GaLore maintains a projection matrix $P_Q \in \mathbb{R}^{h \times r}$ for W_Q and optimizer states in the projected subspace of shape $r \times h$. It holds that:

- GaLore needs to store gradient, projection matrix and optimizer state of each layer’s W_Q , which needs $4hrL$ memory storage.
- GaLore needs to store full model parameters, which needs $12h^2L$ memory storage.
- For backward propagation, GaLore needs to store extra activation X , leading to $L(abs^2 + 9bsh)$ activation memory in total.

We obtain the peak memory of GaLore when the activated module is W_Q :

$$L(abs^2 + 9bsh + 12h^2 + 4hr).$$

From the analysis above, we can similarly generalize the memory storage of different modules. Then we obtain the comparison between LoRA, GaLore and Module-wise BCD.

Table 16: **Peak memory comparison between LoRA, GaLore and Module-wise BCD**

Activating Module	LoRA	GaLore	Modulewise-BCD
W_Q	$L(abs^2 + 9bsh + 12h^2 + 8hr)$	$L(abs^2 + 9bsh + 12h^2 + 4hr)$	$L(abs^2 + 8bsh + 12h^2) + bsh + 3h^2$
W_K	$L(abs^2 + 9bsh + 12h^2 + 8hr)$	$L(abs^2 + 9bsh + 12h^2 + 4hr)$	$L(abs^2 + 8bsh + 12h^2) + bsh + 3h^2$
W_V	$L(abs^2 + 9bsh + 12h^2 + 8hr)$	$L(abs^2 + 9bsh + 12h^2 + 4hr)$	$L(abs^2 + 8bsh + 12h^2) + bsh + 3h^2$
W_O	$L(abs^2 + 9bsh + 12h^2 + 4hr)$	$L(abs^2 + 9bsh + 12h^2 + 8hr)$	$L(abs^2 + 8bsh + 12h^2) + bsh + 3h^2$
W_1	$L(abs^2 + 9bsh + 12h^2 + 20hr)$	$L(abs^2 + 9bsh + 12h^2 + 13hr)$	$L(abs^2 + 8bsh + 12h^2) + bsh + 12h^2$
W_2	$L(abs^2 + 12bsh + 12h^2 + 20hr)$	$L(abs^2 + 12bsh + 12h^2 + 13hr)$	$L(abs^2 + 8bsh + 12h^2) + 4bsh + 12h^2$
All	$L(abs^2 + 15bsh + 12h^2 + 72hr)$	$L(abs^2 + 15bsh + 12h^2 + 42hr)$	$L(abs^2 + 8bsh + 12h^2) + 7bsh + 36h^2$

E.4 Memory analysis of MISA

Let x , y , and z denote the number of activated modules for $W_Q/W_K/W_V$, W_1 , and W_2 , respectively. Based on Table 15, we derive the memory estimation for MISA as follows:

- **Activation Memory**

Compared to layer-wise optimization, MISA requires additional activation storage of:

$$bshx + bshy + 4bshz$$

- **Parameter Memory**

The full model parameters require:

$$12h^2L$$

- **Optimizer States and Gradients**

The memory needed for storing optimizer states and gradients of activated modules is:

$$3h^2x + 12h^2y + 12h^2z$$

Given a trainable parameter ratio threshold δ , where the number of trainable parameters is $12h^2L\delta$, we formulate the optimization problem to determine the maximum memory consumption M :

$$\begin{aligned} \text{Constraint: } & h^2x + 4h^2(y + z) \leq 12h^2L\delta \\ \text{Objective: } & M = L(abs^2 + 8bsh + 12h^2) + bsh(x + y + 4z) \\ & + h^2(3x + 12y + 12z) \end{aligned}$$

The optimal solution yields the peak memory consumption of MISA:

$$L(abs^2 + 8bsh + 12h^2 + 12bsh\delta + 36h^2\delta) \quad (14)$$

E.5 Memory analysis of MISA’s indicators

In this subsection, we denote B be the total number of modules.

According to equation 4, MISA needs to maintain G_b^n, p_b^n .

- **Memory of G_b^n :** For the frozen block b_0 , G_0^n follows the previous step. For the unfrozen block b , since its gradient has been saved, we only need to store $\frac{1}{T} \sum_{t=1}^T \|g_b^{n,t}\|^2$, which is negligible. Therefore, the total memory overhead is $O(B)$.
- **Memory of p_b^n :** According to the definition of p_b^n in equation 4, the previous one p_b^{n-1} can be discarded at n -th sampling period. Therefore, the memory occupation is $O(B)$.

Therefore, the storage of the importance indicator $O(B)$ is negligible compared with the gradients, which is $O(h^2L)$.

Memory efficiency even as model scales. The memory overhead of storing the smoothed historical gradient norm G_b is $O(B)$, which is negligible compared to the $O(h^2L)$ overhead of the gradients themselves (with a relative ratio of $O(1/h^2)$ when $L = O(B)$, as in LLaMA models). As model dimensions h and L increase, this ratio decays quadratically, making the overhead of storing G_b negligible.

E.6 Peak memory comparison between MISA and layer-wise method

Lemma 4. *Let δ be the trainable parameter ratio threshold, when $\delta < \frac{7bs+36h}{12bsL+36hL}$, MISA is more memory efficient than layer-wise method.*

Proof. From conclusion (14), we only need to prove

$$L(abs^2 + 8bsh + 12h^2 + 12bsh\delta + 36h^2\delta) < L(abs^2 + 8bsh + 12h^2) + 7bsh + 36h^2$$

$\iff$

$$\delta < \frac{7bs + 36h}{12bsL + 36hL}$$

It’s obvious that when $\delta < \frac{1}{L}$, memory of MISA is always smaller than layer-wise method. $\square$

E.7 Peak memory comparison between layer-wise method and LoRA

Lemma 5. *As sequence length gets larger, layer-wise optimization method will become more memory efficient than LoRA and Galore. When $s > \frac{36h-72rL}{7bL-7b}$, peak memory of layer-wise optimization method is always smaller than LoRA and Galore.*

Proof. In this proof, we only consider the case when LoRA target and Galore target are all modules.

From table 16, when peak memory of layer-wise method is smaller than LoRA and Galore, we have

$$L (abs^2 + 8bsh + 12h^2) + 7bsh + 36h^2 < L (abs^2 + 15bsh + 12h^2 + 42hr)$$

$\Longleftrightarrow$

$$7bsh + 36h^2 < 7bshL + 42rhL$$

$\Longleftrightarrow$

$$36h - 42rL < s (7bL - 7b)$$

$\Longleftrightarrow$

$$s > \frac{36h - 42rL}{7bL - 7b}$$

□

E.8 Paras/Peak-Memory comparison between layer-wise method and LoRA

Lemma 6. *In the memory-consistent case, layer-wise optimization method can update more parameters than LoRA when the inequality holds $h > \frac{3rL}{2}$.*

Proof. From table 16, we only need to prove

$$\frac{18rhL}{L (abs^2 + 15bsh + 12h^2 + 72rh)} < \frac{12h^2}{L (abs^2 + 8bsh + 12h^2) + 7bsh + 36h^2}$$

$\Longleftrightarrow$

$$3r [L (abs^2 + 8bsh + 12h^2) + 7bsh + 36h^2] < 2h (abs^2 + 15bsh + 12h^2 + 72rh)$$

$\Longleftrightarrow$

$$abs^2 (2h - 3rL) + bsh [30h - 24rL - 21r] + h^2 (36r + 24h - 36rL) > 0$$

When $r < \frac{2h}{3L}$ holds, each part of the last inequality is greater than 0, which completes the proof. □

F Computation Analysis

In this subsection, we adopt the same notation and setting as those in the memory analysis. It is well known that subspace optimization algorithms, such as LoRA, are not computationally efficient. Therefore, in this section, we will only analyze the computational complexity of layer-wise and module-wise optimization methods. Here we present the computation analysis in detail. We will analyze the exact floating point operations (FLOPs) required for **backward propagation** through a single Transformer layer. To begin with, we should remind that the matrices production with size $m \times n$ and $n \times r$ need $2mnr$ FLOPs.

From the forward and backward propagation flows and analysis in E.1, We can estimate the computational overhead under layer-wise setting.

Let:

- b : batch size
- s : sequence length
- h : hidden dimension
- a : number of attention heads
- $d = h/a$: per-head dimension

$$W_Q, W_K, W_V \in \mathbb{R}^{h \times h}, \quad W_O \in \mathbb{R}^{h \times h},$$

$$W_1 \in \mathbb{R}^{h \times 4h}, \quad W_2 \in \mathbb{R}^{4h \times h}$$

Backward FLOPs per layer

1. Feed-Forward Network (FFN)

$$\text{Grad } \partial W_2 : 2bs \cdot 4h \cdot h = 8bsh^2$$

$$\text{Grad } \partial Z_1 : 2bs \cdot h \cdot 4h = 8bsh^2$$

$$\text{Grad } \partial W_1 : 2bs \cdot h \cdot 4h = 8bsh^2$$

$$\text{ReLU mask} : 4bsh$$

$$\text{Total} : \boxed{24bsh^2 + 4bsh}$$

2. Output Projection W_O

$$\text{Grad } \partial W_O : 2bsh^2$$

$$\text{Grad } \partial O : 2bsh^2$$

$$\text{Total} : \boxed{4bsh^2}$$

3. Q/K/V Projection

$$\text{Each of } \partial W_Q, \partial W_K, \partial W_V : 2bsh^2$$

$$\text{Total} : \boxed{6bsh^2}$$

4. Attention Score and Softmax

$$QK^\top : 2bs^2h$$

$$\text{Grad w.r.t } Q, K : 2bs^2h$$

$$\text{Softmax backward} : 2bas^2$$

$$\text{Total} : \boxed{4bs^2h + 2bas^2}$$

5. Attention Output Context (SV)

$$S^\top \partial O : 2bs^2h$$

$$\partial SV^\top : 2bs^2h$$

$$\text{Total} : \boxed{4bs^2h}$$

6. Layer-Norm Backward

$$\boxed{10bsh}$$

Total Backward FLOPs (One Layer)

$$34bsh^2 + 8bs^2h + 2bas^2 + 14bsh$$

F.1 Computation analysis of layer-wise optimization methods

We assume only one layer is unfrozen at each iteration under a block coordinate descent (BCD) strategy.

Activated layer

For an activated layer, we cannot omit the computation cost of the entire layer during backpropagation.

Thus, the total FLOPs without weight gradient computations is:

$$34bsh^2 + 8bs^2h + 2bas^2 + 14bsh$$

Frozen layer

For a frozen layer, we can omit part of the computational cost of calculating the weight gradients during backpropagation.

We exclude the following terms:

- W_Q, W_K, W_V : $6bsh^2$
- W_O : $2bsh^2$
- W_1, W_2 : $16bsh^2$

Thus, the total FLOPs without weight gradient computations is:

$$10bsh^2 + 8bs^2h + 2bas^2 + 14bsh$$

Total computation cost

$$(L - 1)(10bsh^2 + 8bs^2h + 2bas^2 + 14bsh) + 34bsh^2 + 8bs^2h + 2bas^2 + 14bsh$$

F.2 Computation analysis of module-wise optimization methods

Let the total number of activated modules W_Q, W_K, W_V, W_O be x , number of modules W_1, W_2 be y , the expectation of computation cost be C , from analysis of [F](#), we obtain the simple optimization Problem.

$$h^2x + 4h^2y \leq 12h^2L\delta$$

$$C = L(10bsh^2 + 8bs^2h + 2bas^2 + 14bsh) + 2bsh^2x + 8bsh^2y$$

We obtain

$$C_{max} = L(10bsh^2 + 8bs^2h + 2bas^2 + 14bsh) + 24bsh^2L\delta$$

It is obvious that when $\delta < \frac{1}{L}$, module-wise methods demonstrates greater computational efficiency compared to layer-wise optimization methods.

F.3 Computation analysis of MISA’s indicators

In implementation, we first update G_b^n and then compute p_b^n according to Equation (4).

- **Computation overhead of G_b^n :** The dominant term of the computation cost is the calculation for $\|g_b^{n,t}\|^2$, which needs $O(h^2)$ flops.
- **Computation overhead of p_b^n :** The computation overhead of p_b^n only depends on the number of blocks B , which needs $O(B)$ flops.

Therefore, the computational overhead of the importance indicators $O(B + h^2)$ is negligible compared with the gradients, which is $O(bsh^2)$.

Computational efficiency even as model scales. The computation of the smoothed historical gradient norm G_b introduces an overhead of $O(h^2)$. In contrast, the computational overhead for gradients is $O(bsh^2)$, yielding a relative ratio of $O(1/b)$. As the model scales up, (i.e., with increasing h and L), this ratio remains stable, making the overhead of computing G_b ’s overhead negligible.

G Broader Impacts

This paper presents work that aims to advance the field of machine learning. The proposed method, MISA, is designed to efficiently train LLMs with low memory consumption. This will facilitate further research regarding memory-efficient LLM training in the future. We believe that this work will not cause significant societal consequences.

H Experimental Hyperparameters

For all baselines and our proposed method, we conducted extensive hyperparameter searches. The learning rate was searched in $\{2e-4, 1e-4, 5e-5, 1e-5, 5e-6, 3e-6, 1e-6\}$. For LoRA and GaLore, we explored ranks in $\{8, 16, 32\}$, and we found that a rank of 16 or 32 consistently yielded better performance than 8. For MISA, η was searched in $\{0.1, 0.5, 1\}$. The table below presents the optimal hyperparameter settings.

The batch size in the following tables is represented as micro batch size $\times$ gradient accumulation.

H.1 Hyperparameters for commonsense reasoning

Hyperparameters	LLaMA3-8B		Qwen2.5-7B	
	LoRA	DoRA	LoRA	DoRA
Rank	32	16	32	16
α	64	32	64	32
Dropout		0.05		
Optimizer		AdamW		
Learning rate	1e-4	1e-4	2e-4	2e-4
Batch size		4 $\times$ 4		
Warmup Steps		100		
Epochs		3		
Target module	$W_q, W_k, W_v, W_{up}, W_{down}$			

Table 17: Hyperparameters of LoRA and DoRA in commonsense reasoning tasks.

Hyperparameters	LLaMA3-8B			Qwen2.5-7B		
	MISA	BAdam	LISA	MISA	BAdam	LISA
Optimizer			AdamW			
Learning rate	1e-5	5e-6	3e-6	5e-6	5e-6	3e-6
Batch size			4×4			
Warmup Steps			0			
Activated parameters	1%, 3%	1 layer	1 layer	1%, 3%	1 layer	1 layer
MISA's η	1	-	-	1	-	-
T			50			
Epochs			3			

Table 18: Hyperparameters of MISA, BAdam and LISA in commonsense reasoning tasks.

H.2 Hyperparameters for math reasoning

Hyperparameters	LLaMA3-8B		Qwen2.5-7B	
	LoRA	DoRA	LoRA	DoRA
Rank	32	32	32	32
α		32		
Dropout		0.05		
Optimizer		AdamW		
Learning rate	2e-4	2e-4	1e-4	1e-4
Batch size		4×1		
Warmup Steps		0		
Epochs		3		
Target module		$W_q, W_v, W_{up}, W_{down}$		

Table 19: Hyperparameters of LoRA and DoRA in math reasoning tasks.

Hyperparameters	LLaMA3-8B			Qwen2.5-7B		
	MISA	BAdam	LISA	MISA	BAdam	LISA
Optimizer			AdamW			
Learning rate	5e-6	5e-6	5e-6	1e-5	1e-5	5e-6
Batch size			4×1			
Warmup Steps			0			
Activated parameters	1%, 3%	1 layer	1 layer	1%, 3%	1 layer	1 layer
MISA's η	0.5	-	-	1	-	-
T			50			
Epochs			3			

Table 20: Hyperparameters of MISA, BAdam and LISA in math reasoning tasks.

H.3 Hyperparameters for instruction fine-tuning

Hyperparameters	LLaMA2-7B		Mistral-7B		Tiny-LLaMA	
	LoRA	GaLore	LoRA	GaLore	LoRA	GaLore
Rank	32	32	32	32	32	32
α	64	64	64	64	64	64
Dropout	0.05	0	0.05	0	0.05	0
Optimizer	AdamW					
Learning rate	2e-4	3e-6	1e-4	1e-6	2e-4	1e-5
Batch size	2 $\times$ 8					
Warmup Steps	0					
Epochs	3					
Target module	$W_q, W_k, W_v, W_o, W_{up}, W_{gate}, W_{down}$					

Table 21: Hyperparameters of LoRA and DoRA on Alpaca GPT4.

Hyperparameters	LLaMA2-7B			Mistral-7B			Tiny-LLaMA		
	MISA	BAdam	LISA	MISA	BAdam	LISA	MISA	BAdam	LISA
Optimizer	AdamW								
Learning rate	1e-5	1e-5	5e-6	1e-5	5e-6	1e-5	5e-5	1e-5	1e-5
Batch size	2 $\times$ 8								
Warmup Steps	0								
Activated parameters	3%	1 layer	1 layer	3%	1 layer	1 layer	4.545%	1 layer	1 layer
MISA’s η	0.5	-	-	1	-	-	0.5	-	-
T	50								
Epochs	3								

Table 22: Hyperparameters of MISA, BAdam and LISA on Alpaca GPT4.

H.4 Hyperparameters for pre-training

Hyperparameters	MISA
Optimizer	AdamW
Learning rate	1e-3
Batch size	32 $\times$ 256
Warmup Steps	0
MISA’s δ	3%, 25%
MISA’s η	300
T	50
Sequence Length	256
Training Steps	52000

Table 23: Hyperparameters of MISA on pre-training LLaMA2 350M.

Hyperparameters	GaLore
Optimizer	AdamW
Learning rate	1e-3
Batch size	32 $\times$ 256
Warmup Steps	0
GaLore’s rank	32, 256
GaLore’s α	1
T	50
Sequence Length	256
Training Steps	52000

Table 24: Hyperparameters of GaLore on pre-training LLaMA2 350M.